



MEDIATEK

# Android 15 Camera Legacy Sensor Driver Bring-Up Guide

(Support MT8365, MT8371, MT8385, MT8391, MT8766,  
MT8768, MT8786, MT8788, MT8781, MT8791)

**Version:** 1.7  
**Release date:** 2024-03-27

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

---

MT8365, MT8371, MT8385, MT8391, MT8766, MT8768, MT8786, MT8788, MT8781, MT8791

## Android 15 Camera Legacy Sensor Driver Bring-Up Guide

## Version History

| Version | Date       | Description                                                                                                                                                                                                                                                                                                                                                         |
|---------|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.0     | 2020-08-01 | Initial draft                                                                                                                                                                                                                                                                                                                                                       |
| 1.1     | 2020-10-01 | Official release                                                                                                                                                                                                                                                                                                                                                    |
| 1.2     | 2021-10-01 | <p>p55 to p58<br/>Update some METADATA information<br/>Exposure / Field of View / sensor size / test pattern</p> <p>p49 to p50<br/>How to add a tuning file</p> <p>p44, p30<br/>Add a new command in feature_control<br/>SENSOR_FEATURE_GET_GAIN_RANGE_BY_SCENARIO<br/>SENSOR_FEATURE_GET_BASE_GAIN_ISO_AND_STEP<br/>SENSOR_FEATURE_GET_MIN_SHUTTER_BY_SCENARIO</p> |
| 1.3     | 2023-05-30 | Official release                                                                                                                                                                                                                                                                                                                                                    |
| 1.4     | 2023-07-07 | Renew document template                                                                                                                                                                                                                                                                                                                                             |
|         | 2024-04-18 | Modify it for Android U                                                                                                                                                                                                                                                                                                                                             |
| 1.4.1   | 2024-05-24 | Modify for the LD2.0 series                                                                                                                                                                                                                                                                                                                                         |
| 1.4.2   | 2024-07-10 | Modify grammatical description                                                                                                                                                                                                                                                                                                                                      |
| 1.5.0   | 2024-12-20 | Modify for Kernel 6.6, Android 15 architecture                                                                                                                                                                                                                                                                                                                      |
| 1.6     | 2025-02-20 | Modify to make this document for MT8766, MT8768, MT8786, MT8781, MT8791                                                                                                                                                                                                                                                                                             |
| 1.7     | 2025-03-27 | <p>Add MT8365, MT8371, MT8385, MT8191, and MT8788 platform</p> <p>Add mapping table of platform and s/w code path to chapter 6</p> <p>Fix some typo</p>                                                                                                                                                                                                             |

## Terminology

| Abbreviation | Description                                       |
|--------------|---------------------------------------------------|
| 3A           | Auto Exposure, Auto White Balance, and Auto Focus |
| CMOS         | Complementary Metal-Oxide-Semiconductor           |
| CRC          | Cyclic Redundancy Check                           |
| CSI-2        | Camera Serial Interface 2                         |
| CTS          | Can Transmit Data                                 |
| DT           | Device Tree                                       |
| GPIO         | General Purpose Inputs-Outputs                    |
| HAL          | Hardware Abstraction Layer                        |
| HDR          | High Dynamic Range                                |
| HL           | Head Listening                                    |
| I/O          | Input/Output                                      |
| ID           | Identification                                    |
| IoT          | Internet of Things                                |
| ISP          | Image Signal Processor                            |
| LSC          | Land-Side Capacitors                              |
| MIPI         | Mobile Industry Processor Interface               |
| MOL          | MediaTek On-Line                                  |
| P1           | ISP Pass1                                         |
| P2           | ISP Pass2                                         |
| PCLK         | APB Clock                                         |
| PMIC         | Power Management Integrated Chip                  |
| SCL          | Serial Clock Line                                 |
| SDA          | Serial Data                                       |
| sync         | Synchronize; Synchronization                      |
| TG           | Target                                            |
| Vsync        | Vertical Synchronization                          |

## Table of Contents

|                                                         |           |
|---------------------------------------------------------|-----------|
| <b>Version History</b>                                  | <b>2</b>  |
| <b>Terminology</b>                                      | <b>3</b>  |
| <b>Table of Contents</b>                                | <b>4</b>  |
| <b>List of Figures</b>                                  | <b>5</b>  |
| <b>List of Tables</b>                                   | <b>5</b>  |
| <b>1 Introduction</b>                                   | <b>6</b>  |
| 1.1 Purpose                                             | 6         |
| 1.2 Scope                                               | 6         |
| <b>2 Camera Sensor Basic Knowledge</b>                  | <b>7</b>  |
| 2.1 About This Chapter                                  | 7         |
| 2.1.1 Linux Kernel                                      | 7         |
| 2.1.2 Device Tree (DT)                                  | 7         |
| 2.1.3 Inter-Integrated Circuit (I2C)                    | 7         |
| 2.1.4 MIPI CSI-2                                        | 7         |
| 2.1.5 RAW and YUV                                       | 8         |
| 2.2 Interactions with the Camera Sensor                 | 8         |
| <b>3 Overview</b>                                       | <b>10</b> |
| 3.1 Sensor Driver Architecture                          | 10        |
| <b>4 Hardware Layout</b>                                | <b>13</b> |
| <b>5 File Preparation</b>                               | <b>15</b> |
| <b>6 Sensor Driver Porting</b>                          | <b>16</b> |
| 6.1 Configure Hardware Related (PDN/RST/MCLK/I2C/Power) | 16        |
| 6.2 Modify Compilation Options Related: defconfig       | 18        |
| 6.3 Add a new sensor ID                                 | 19        |
| 6.4 Add New Sensor to Sensor List                       | 20        |
| 6.5 Source Code Setting                                 | 21        |
| 6.5.1 How to Change the Sensor Driver File              | 21        |
| 6.5.1.1 static kal_uint32 open                          | 21        |
| 6.5.1.2 static kal_uint32 get_info()                    | 24        |
| 6.5.1.3 static kal_uint32 get_resolution                | 27        |
| 6.5.1.4 static kal_uint32 feature_control               | 27        |
| 6.5.1.5 static kal_uint32 control                       | 39        |
| 6.5.1.6 static kal_uint32 close                         | 40        |
| <b>7 User Space Porting</b>                             | <b>41</b> |
| 7.1 Modify ProjectConfig.mk                             | 41        |
| 7.2 Modify CameraConfig.mk                              | 41        |
| 7.3 Modify device-camera.mk                             | 41        |
| 7.4 Add a New Sensor to mtkcamvars.go                   | 42        |
| 7.5 Config Hardware Related (mclk CSI)                  | 42        |
| 7.6 Add New Sensor ID and Name                          | 43        |

## Android 15 Camera Legacy Sensor Driver Bring-Up Guide

|         |                                                      |           |
|---------|------------------------------------------------------|-----------|
| 7.7     | Add New Sensor to Sensor List .....                  | 44        |
| 7.8     | Add Tuning/Metadata Files .....                      | 45        |
| 7.8.1   | How to Prepare for Your Tuning File .....            | 45        |
| 7.8.2   | How to Prepare for Your Metadata File .....          | 47        |
| 7.8.2.1 | MTK_SCALER_AVAILABLE_STREAM_CONFIGURATIONS .....     | 47        |
| 7.8.2.2 | MTK_SENSOR_INFO_OUTPUT_REGION_ON_ACTIVE_ARRAY .....  | 48        |
| 7.8.2.3 | MTK_SENSOR_INFO_REAL_OUTPUT_SIZE .....               | 48        |
| 7.8.2.4 | MTK_SENSOR_INFO_ORIENTATION .....                    | 49        |
| 7.8.2.5 | MTK_SENSOR_INFO_WANTED_ORIENTATION .....             | 49        |
| 7.8.2.6 | MTK_SENSOR_INFO_FACING .....                         | 49        |
| 7.8.2.7 | MTK_SENSOR_INFO_PIXEL_ARRAY_SIZE .....               | 49        |
| 8       | <b>METADATA Field Associated with a Sensor .....</b> | <b>51</b> |
| 8.1     | Exposure Related .....                               | 51        |
| 8.2     | FOV Related .....                                    | 52        |
| 8.3     | Sensor Size Related .....                            | 52        |
| 8.4     | Test Pattern Mode .....                              | 54        |
| 9       | <b>Unit Test .....</b>                               | <b>55</b> |
| 10      | <b>Integration Test .....</b>                        | <b>56</b> |
|         | <b>Exhibit 1 Terms and Conditions .....</b>          | <b>57</b> |

## List of Figures

|             |                                                                   |    |
|-------------|-------------------------------------------------------------------|----|
| Figure 2-1. | I/O Pins of a MIPI Sensor .....                                   | 7  |
| Figure 2-2. | Parallel and MIPI Camera Interface .....                          | 8  |
| Figure 3-1. | Sensor Driver Architecture .....                                  | 10 |
| Figure 3-2. | Sensor Porting Flow .....                                         | 10 |
| Figure 3-3. | Process of Searching for Sensor .....                             | 11 |
| Figure 3-4. | Sensor Power on & Scenario Setting Flow .....                     | 12 |
| Figure 4-1. | Sensor Module .....                                               | 13 |
| Figure 4-2. | Relationship between Sensor Module and MTK IoT EVK Platform ..... | 14 |
| Figure 6-1. | Setting GPIO Method .....                                         | 17 |
| Figure 6-2. | Power On Sequence .....                                           | 18 |

## List of Tables

|            |                                         |    |
|------------|-----------------------------------------|----|
| Table 4-1. | Sensor Signal Description .....         | 14 |
| Table 6-1. | Mapping Table of MTK IoT Platform ..... | 16 |

# 1 Introduction

## 1.1 Purpose

The guidelines for developing a RAW image sensor are provided in this document.

## 1.2 Scope

This document is based on the MediaTek IoT platform with Android 15.

In this document, we assume the RAW image sensor output “Bayer RAW” image format and do image calibration, quality tuning, and 3A in MediaTek IoT platform ISP.

Applicable platform:

- MT8365 (Genio 350)
- MT8385 (Genio 500)
- MT8371 (Genio 520)
- MT8391 (Genio 720)
- MT8766
- MT8768
- MT8786
- MT8788
- MT8781
- MT8791



## 2 Camera Sensor Basic Knowledge

### 2.1 About This Chapter

This chapter introduces the basic knowledge needed for understanding camera sensor porting.

#### 2.1.1 Linux Kernel

Linux kernel basic knowledge such as how to register a character device driver, is required. For more detailed information, refer to the classic book “Understanding the Linux Kernel” or visit the links below:

<https://linux-kernel-labs.github.io/refs/heads/master/lectures/intro.html>

[http://www.staroceans.org/kernel-and-driver/Understanding\\_Linux\\_Kernel\\_-\\_en.pdf](http://www.staroceans.org/kernel-and-driver/Understanding_Linux_Kernel_-_en.pdf)

#### 2.1.2 Device Tree (DT)

DT background knowledge such as how Linux uses the DT, is required. Refer to “SOP for Linux DTS” on MOL or visit the links below:

[https://elinux.org/Device\\_Tree\\_Reference](https://elinux.org/Device_Tree_Reference)

#### 2.1.3 Inter-Integrated Circuit (I2C)

I2C basic knowledge, such as how to check I2C device slave address (7-bit) as per waveform, is required. Refer to “I2C User Guide” on MOL or visit the links below:

#### 2.1.4 MIPI CSI-2

MIPI CSI-2 basic knowledge is also required. The following shows the I/O pins of a MIPI sensor:

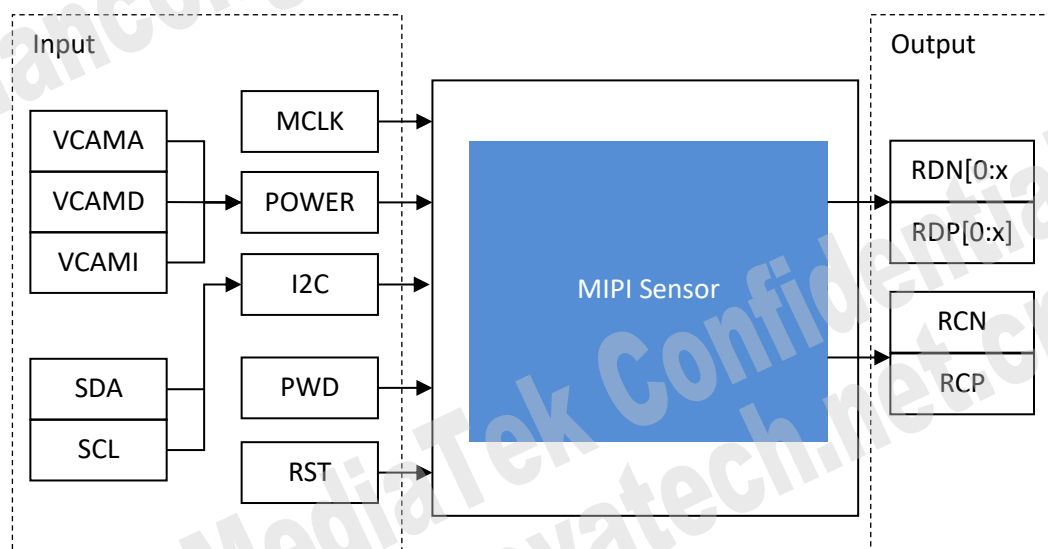


Figure 2-1. I/O Pins of a MIPI Sensor

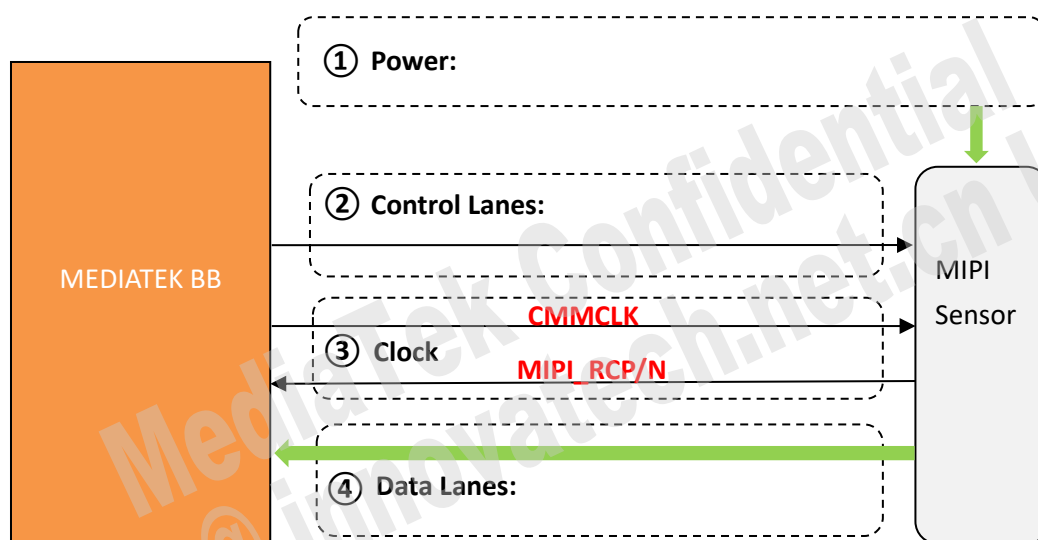
For more details, visit: <https://www.mipi.org/specifications/csi-2>

### 2.1.5 RAW and YUV

Basic knowledge about RAW and YUV such as RAW10, RAW12, YUYV formats, is also required. Please refer to MIPI CSI-2 Specification for more details: <https://www.mipi.org/specifications/csi-2>

## 2.2 Interactions with the Camera Sensor

Camera sensor interacts with Image Signal Processor (ISP), using the I2C bus for control and the CSI-2 bus for data. The following shows an example of a camera hardware connection:



**Figure 2-2. Parallel and MIPI Camera Interface**

Here “BB” represents MediaTek baseband, responding to MediaTek Camera Image Signal Processor (ISP).

#### Dependency Modules:

GPIO, PMIC, I2C, ISP TG MCLK.

Use adb shell to check the module of dependency:

#### - PMIC

Show specific mount information, indicating whether the status is normal.

```
cat /sys/kernel/debug/regulator/regulator_summary
```

#### - GPIO

Display the node and its associated information, and indicate if the status of the General Purpose Input/Output (GPIO) is within normal parameters.



MT8365, MT8371, MT8385, MT8391, MT8766, MT8768, MT8786, MT8788, MT8781, MT8791

## Android 15 Camera Legacy Sensor Driver Bring-Up Guide

```
cat /sys/kernel/debug/pinctrl/*****.pinctrl-pinctrl_paris/pins
```

**I2C**

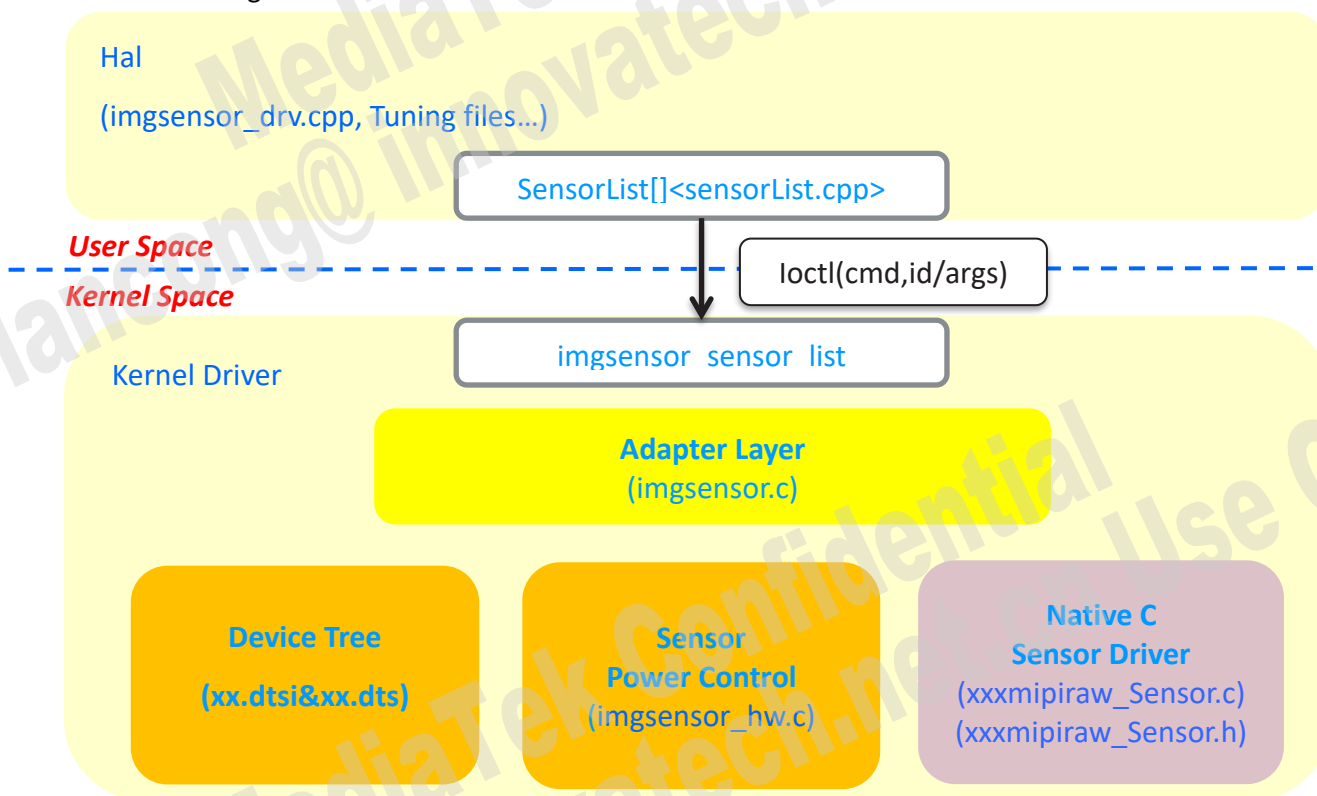
Show the I2C bus node, indicating whether the I2C status is normal.

```
cd /sys/bus/i2c
```

## 3 Overview

### 3.1 Sensor Driver Architecture

The architecture diagram of the sensor driver is as follows:



**Figure 3-1. Sensor Driver Architecture**

- **Imgsensor Drv**
  - User space driver
- **Adapter layer**
  - An adapter layer for the Linux character device driver and native sensor driver
- **Sensor power control**
  - Control the sensor power on/off

Porting sensors means adapting the new sensor to the public version of the architecture. The following Figure 3-2 shows the process of sensor porting.



**Figure 3-2. Sensor Porting Flow**

The following steps are summarized and will be explained in detail in subsequent chapters.

**Hardware layout:** Collect the information on each pin connected to the sensor module.

**File preparation:** Including user space and kernel space files. The user space is for tuning and metadata files, and the kernel space is the driver source code for the sensor.

**Kernel space configuration:** Configure software flow and hardware-related parameters.

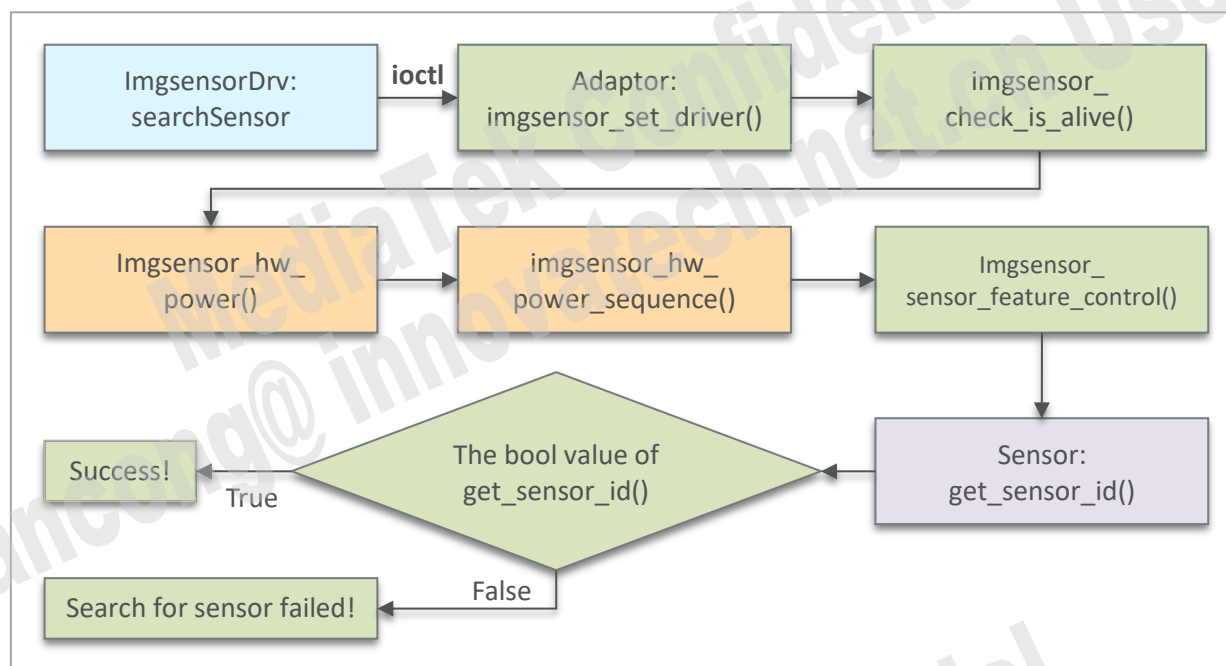
**User space configuration:** Configure software flow and hardware-related parameters.

**Unit Test:** Test and verification after sensor porting. The porting process can be logically divided into two parts. One part is testing whether it is normal for the platform to power and control the sensor module, and the other is testing whether it is normal for the platform to receive data from the sensor.

**Integration Test:** Configure specific information related to the sensor, and data sent by the sensor in the form of pictures, and perform linearity tests on the shutter and gain.

After porting is completed step by step, the sensor's UT is the main job, and the search sensor ID in UT is a key point. Search success often means close to success.

The following is the process of the search sensor:



**Figure 3-3. Process of Searching for Sensor**

It can be seen from the flow that finally, hw\_power\_sequence completes the trigger sensor device and then queries the sensor ID through i2c, then judges the status of the device by returning the result. Next, we will introduce the process of getting raw data out of a sensor. After the search for the sensor ID is successful, the sensor's info will be sent to the receiving end to prepare for data transmission. Next, look at the flow of setting up the sensor scenario.

MT8365, MT8371, MT8385, MT8391, MT8766, MT8768, MT8786, MT8788, MT8781, MT8791

Android 15 Camera Legacy Sensor Driver Bring-Up Guide

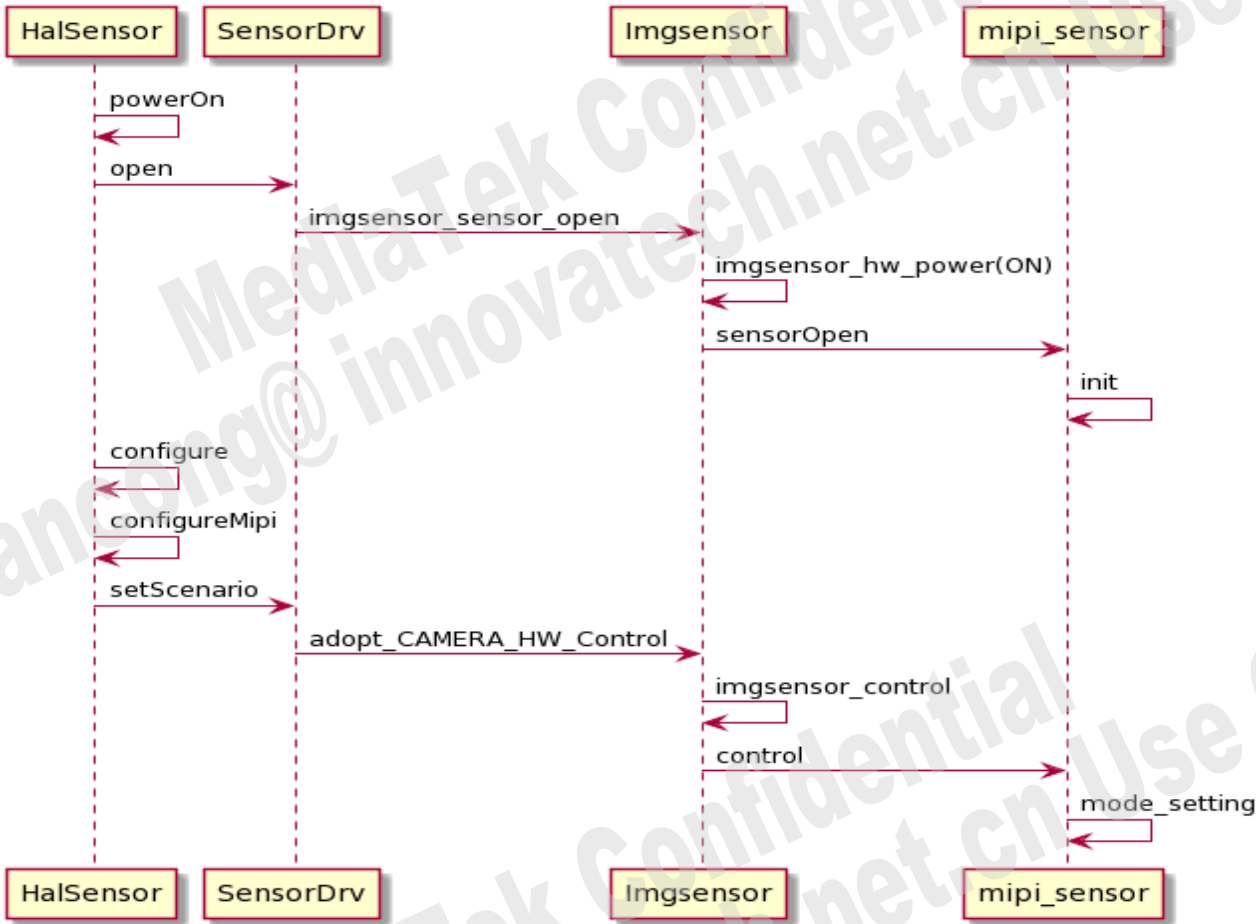


Figure 3-4. Sensor Power on & Scenario Setting Flow

## 4 Hardware Layout

The chapter analyzes the concept and general context of porting and then lists the things that can be prepared beforehand. The following example will illustrate the relationship between the sensor module, camera daughter board, and MTK EVK platform.

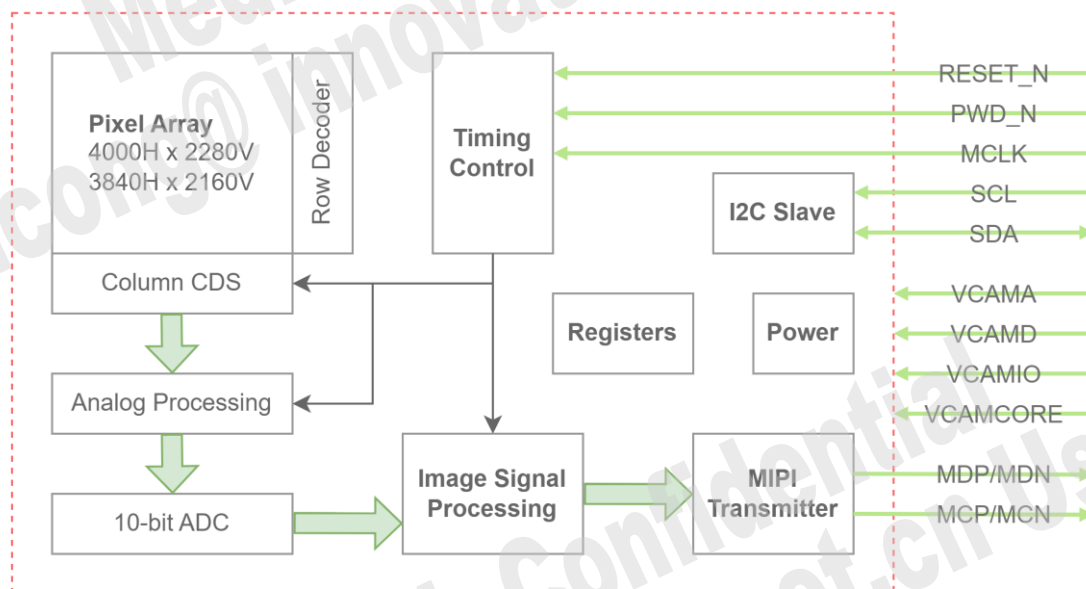


Figure 4-1. Sensor Module

In Figure 4-1, the red area is the sensor module, and the connection between the platform and the sensor is indicated by gray arrows, which can be logically divided into two parts:

1. VCAMA, VCAMD, VCMAIO, VCAMCORE, RESET, PDWN, and MCLK can trigger the sensor, while SDA and SCL can communicate with the sensor and can be used to read the sensor ID.
2. After the registers of the sensor module are set, the sensor starts to output data, and the platform receives image data through the MIPI CSI interface.

In Figure 4-2, we can see the VCAMA, VCAMD, VCAMIO, and VCAMCORE are supplied by LDO/BUCK IC, and they could be controlled by MTK EVK GPIO pins: CAM0\_DVDD\_EN, CAM0\_AVDD\_EN, CAM1\_DVDD\_EN, and CAM1\_AVDD\_EN; the RESET\_N and PWD\_N are controlled by CMMRST and CMMPDN; the master clock is provided by CMMCLK; the I2C SDA and SCL are connected to CAM0\_SDA0 and CAM0\_SCL0; the MIPI CSI2 TX lanes on sensor module are connecting to MDP/MDN and MCP/MCN. We can organize the mapping into Table 4-1.

Android 15 Camera Legacy Sensor Driver Bring-Up Guide

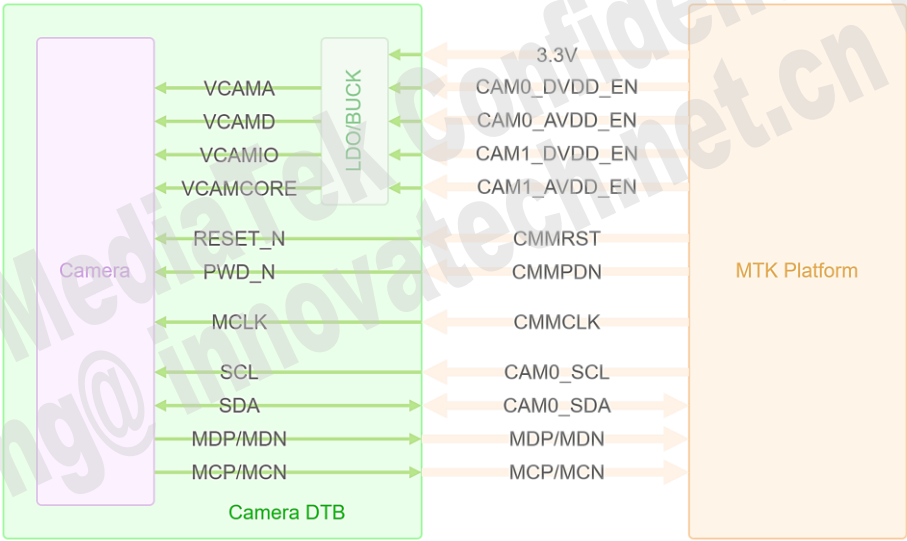


Figure 4-2. Relationship between Sensor Module and MTK IoT EVK Platform

Table 4-1. Sensor Signal Description

| Name               | Type                    | Description                                              | Type     | MTK Platform Pins          |
|--------------------|-------------------------|----------------------------------------------------------|----------|----------------------------|
| VCAMD              | Power supply            | Power for a digital circuit                              | LDO/BUCK | CAM0_DVDD_EN (GPIO60)      |
| VCAMA              | Power supply            | Power for an analog circuit                              | LDO/BUCK | CAM0_AVDD_EN (GPIO56)      |
| VCAMIO             | Power supply            | Power for the digital I/O circuit                        | LDO/BUCK | CAM1_DVDD_EN (GPIO58)      |
| VCAMCORE           | Power supply            | Power for the core circuit                               | LDO/BUCK | CAM1_AVDD_EN (GPIO59)      |
| RESET_N            | Reset pin               | Chip reset control<br>Standby/normal operation           | GPIO     | CMMRST (GPIO55)            |
| PWD_N              | Power down pin          | Sensor power-down control<br>Chip reset/normal operation | GPIO     | CMMPDN (GPIO54)            |
| SDA/SCL            | I2C data/clock          | Read and write sensor registers                          | I2C-bus  | I2C_0<br>CAM0_SCL/CAM0_SDA |
| MCLK               | Master clock            | Sensor system input clock                                | MCLK     | CMMCLK (GPIO22)            |
| MDP/MDN<br>MCP/MCN | MIPI TX data/clock lane | Data transmission                                        | MIPI CSI | CSI_0                      |

Please refer to Chapter 6.1 to configure power control, reset, power down pin, master clock, and MIPI CSI port.



## 5 File Preparation

Hardware confirmation and file preparation can be done in advance to speed up the porting process.

- Sensor datasheet  
Including sensor control and register settings of each mode, provided by sensor vendor FAE.
- Tuning files  
Tuning files include parameter settings related to the sensor's characteristics. In the bring-up sensor phase, you could copy from an existing raw sensor folder before starting to tune the image quality.
- Metadata files  
Including parameters about the sensor output image data, copy based on the sensor used in the public version.

## 6 Sensor Driver Porting

The source code may be located at specific paths in the kernel space and user space, and there may be differences in some keywords in the source code path. Therefore, we provide a mapping in Table 6-1.

Table 6-1. Mapping Table of MTK IoT Platform

| Platform           | {\$SW_CODE_PATH} |
|--------------------|------------------|
| MT8365 (Genio 350) | mt8168           |
| MT8385 (Genio 500) | mt6771           |
| MT8371 (Genio 520) | mt8189           |
| MT8391 (Genio 720) | mt8189           |
| MT8766             | mt6761           |
| MT8768             | mt6765           |
| MT8786             | mt6769           |
| MT8788             | mt6771           |
| MT8781             | mt6789           |
| MT8791             | mt6877           |

### 6.1 Configure Hardware Related (PDN/RST/MCLK/I2C/Power)

**Description:**

Each pin of the sensor should be configured, with the public version taken as an example.

**Path:**

**v6.1 and later:**

```
kernel/kernel_device_modules-XX/arch/arm64/boot/dts/mediatek/{$Project}.dts
```

**v5.15 and before:**

```
kernel-XX/arch/arm64/boot/dts/mediatek/{$Project}.dts
```

**Example:**

```
kernel/kernel_device_modules-6.6/arch/arm64/boot/dts/mediatek/aiot8863p2_64_bsp.dts
```

The camera related device tree nodes had been moved to `cust_{$Project}_camera.dtsi` file. Please confirm the path and file name in `{$project}.dts` and trace all the camera related include files. The following are examples:

*aiot8863p2\_64\_bsp.dts*

```
11
12     #include "mediatek/cust_mt6835_camera.dtsi"
13
```

*cust\_mt6835\_camera.dtsi*

```
5
6     #include "mediatek/cust_mt6835_camera_common.dtsi"
```

cust\_mt6835\_camera\_common.dtsi

```
pinctrl-names = "default",
    "cam0_rst0", "cam0_rst1",
    "cam0_pnd0", "cam0_pnd1",
    "cam1_rst0", "cam1_rst1",
    "cam1_pnd0", "cam1_pnd1",
    "cam2_rst0", "cam2_rst1",
    "cam2_pnd0", "cam2_pnd1",
    "cam0_mclk_off",
    "cam0_mclk_2mA", "cam0_mclk_4mA",
    "cam0_mclk_6mA", "cam0_mclk_8mA",
    ...
```

Name for  
pinctrl\_lookup\_state()

```
pinctrl-0 = <camera_pins_default>;
pinctrl-1 = <camera_pins_cam0_rst_0>;
pinctrl-2 = <camera_pins_cam0_rst_1>;
pinctrl-3 = <camera_pins_cam0_pnd_0>;
pinctrl-4 = <camera_pins_cam0_pnd_1>;
```

```
&pio {
    /* CAMERA GPIO standardization */
    camera_pins_cam0_rst_0: cam0@0 {
        pins_cmd_dat {
            pinmux = <PINMUX_GPIO111_FUNC_GPIO111>;
            slew-rate = <1>;
            /*direction out used only. output_low or high*/
            output-low;
        };
    };
};
```

Set GPIO111 Mode

GPIO Output Low

**Figure 6-1. Setting GPIO Method**

The above shows the specific configuration in dts, which is filled according to the Linux dts specification, and finally selects the state of GPIO by calling the standard interface pinctrl\_lookup\_state(), the above example shows that "cam0\_rst0" corresponds to the output low level of GPIO111. Complete the rst pdn mclk according to the information in Table 4-1.

For the power on/off sequence, reference the specific sensor datasheet.

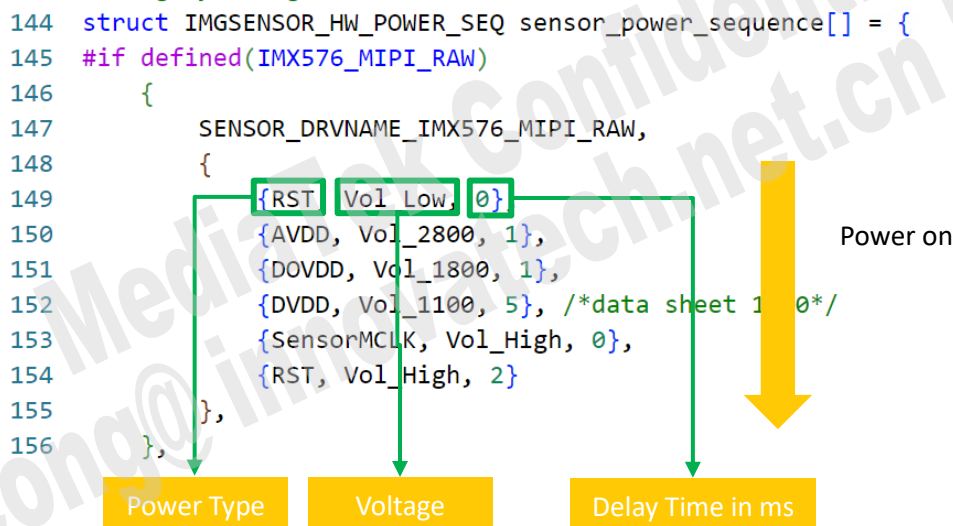


Figure 6-2. Power On Sequence

The new porting requires filling in the sensor for the current one based on the power-on sequence specified in the datasheet.

Note: Incorrect sequence will result in sensor ID search fail. Sometimes it may not be read probabilistically. HW is required to measure the timing here, and it must strictly comply with the spec provided by the vendor.

If power sequence only selects the first open config sequence, please check `imgsensor_hw.c` `imgsensor_hw_power_sequence` API, whether there is following HL code before while loop power on. If not have, please add it.

```

180 while (ppwr_seq < ppower_sequence + IMGSSENSOR_HW_SENSOR_MAX_NUM &&
181 ppwr_seq->name != NULL) {
182     if (!strcmp(ppwr_seq->name, PLATFORM_POWER_SEQ_NAME)) {
183         if (sensor_idx == ppwr_seq->idx)
184             break;
185     } else {
186         if (!strcmp(ppwr_seq->name, pcurr_idx))
187             break;
188     }
189     ppwr_seq++;
190 }

```

## 6.2 Modify Compilation Options Related: defconfig

### Description:

defconfig is a macro-option for compilation, adding the newly added sensor file to the compilation.

### Path:

#### v6.1 and later:

```

kernel/kernel_device_modules-XX/arch/arm64/configs/{KRN_TARGET_PROJECT}_defconfig
kernel/kernel_device_modules-XX/kernel/configs/{$SW_CODE_PATH}_overlay.config

```

#### v5.15 and before:

```

kernel-XX/arch/arm64/configs/{KRN_TARGET_PROJECT}_defconfig

```

The `{{SW_CODE_PATH}}_overlay.config` is used to overlay the settings of `{KRN_TARGET_PROJECT}_defconfig`. Please add your sensor to `CONFIG_CUSTOM_KERNEL_IMGSENSOR` in the `defconfig` or `overlay.config` file. In the LD2.0 architecture, you can identify the `defconfig` file name through `{KRN_TARGET_PROJECT}` in `vnd_{Project}.mk` and then `KERNEL_DEFCONFIG` in `krn_{KRN_TARGET_PROJECT}.mk`; also `{{SW_CODE_PATH}}` in `BoardConfig-vext.mk`.

Example:

`vnd_aiot8863p2_64_bsp.mk`

```
KRN_TARGET_PROJECT := mgk_64_k66
KRN_TARGET_PROJECT_FOLDER := device/mediatek/kernel/${KRN_TARGET_PROJECT}
include ${KRN_TARGET_PROJECT_FOLDER}/krn_${KRN_TARGET_PROJECT}.mk
```

`krn_mgk_64_k66.mk`

```
KERNEL_DEFCONFIG ?= mgk_64_k66_defconfig
```

`mgk_64_k66_defconfig`

```
CONFIG_CUSTOM_KERNEL_IMGSENSOR="imx214_mipi_raw imx256_mipi_raw ov13855_mipi_raw"
```

**Note:** The configuration of `CONFIG_CUSTOM_KERNEL_IMGSENSOR` should be consistent with the user layer, including the name and sequence. The user layer and the kernel layer will be checked for matching during compilation.

### 6.3 Add a new sensor ID

Description:

Include the sensor model and specific name for identifying the porting sensor.

Path:

**v6.1 and later:**

```
kernel/kernel_device_modules-XX/drivers/misc/mediatek/imgsensor/inc/kd_imgsensor.h
```

**v5.15 and before:**

```
kernel-XX/drivers/misc/mediatek/imgsensor/inc/kd_imgsensor.h
```

Config sensor ID:

```
#define OV13855_SENSOR_ID 0xD855
```

Define the sensor device driver name:

```
#define SENSOR_DRVNAME_OV13855_MIPI_RAW "ov13855mipiraw"
```

The sensor ID is determined from the datasheet:

|        |         |      |   |                         |
|--------|---------|------|---|-------------------------|
| 0x300B | CHIP ID | 0xD8 | R | Bit[7:0]: chip_id[15:8] |
| 0x300C | CHIP ID | 0x55 | R | Bit[7:0]: chip_id[7:0]  |

Take OV13855 as an example. Here is the value of the sensor register. We are comparing the defined sensor ID with the value read from the sensor register. The result is the same, and it returns successfully.

6.4 Add New Sensor to Sensor List

Description:

To add the porting sensor to the sensor Adapter, it should first be declared in the header file for calling. Add a new sensor initialization function declaration, e.g.:

Path:

v6.1 and later:

```
kernel/kernel_device_modules-XX/drivers/misc/mediatek/imgsensor/src/common/v1_1/
imgsensor_sensor_list.h
```

v5.15 and before:

```
kernel-XX/drivers/misc/mediatek/imgsensor/src/common/v1_1/imgsensor_sensor_list.h
```

```
UINT32 OV13855_MIPI_RAW_SensorInit(struct SENSOR_FUNCTION_STRUCT **pfFunc);
```

Add a new sensor to the kernel sensor list, e.g.:

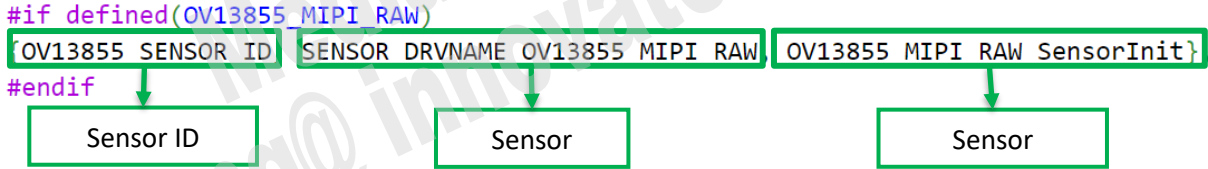
Path:

v6.1 and later:

```
kernel/kernel_device_modules-XX/drivers/misc/mediatek/imgsensor/src/common/v1_1/
imgsensor_sensor_list.c
```

v5.15 and before:

```
kernel-XX/drivers/misc/mediatek/imgsensor/src/common/v1_1/imgsensor_sensor_list.c
```



**Note:** The sensor ID sensor name is the same as `kd_imgsensor.h`, and a new definition is declared for the new sensor.

## 6.5 Source Code Setting

### 6.5.1 How to Change the Sensor Driver File

#### ■ Sensor driver Relevant API

The driver file is in the path as below:

v6.1 and later:

kernel/kernel\_device\_modules-XX/drivers/misc/mediatek/imgsensor/src/common/v\*/gc\*\*\*\*mipiraw/

v5.15 and before:

kernel-XX/drivers/misc/mediatek/imgsensor/src/common/v\*/gc\*\*\*\*mipiraw/

| Relevant API Name                 | Description                         |
|-----------------------------------|-------------------------------------|
| <a href="#">open()</a>            | Initial the sensor                  |
| <a href="#">get_info()</a>        | Get sensor info from the driver     |
| <a href="#">get_resolution()</a>  | Get sensor resolution               |
| <a href="#">feature_control()</a> | The sensor feature control function |
| <a href="#">control()</a>         | Control the sensor mode             |
| <a href="#">close()</a>           | Uninitialed the sensor              |

| API in feature_control                            | Description                                     |
|---------------------------------------------------|-------------------------------------------------|
| <a href="#">set_shutter</a>                       | Set the shutter to register                     |
| <a href="#">set_dummy</a>                         | Update frame length & line length               |
| <a href="#">gain2reg</a>                          | Mapping the gain value and registering the gain |
| <a href="#">set_auto_flicker_mode</a>             | Enable/disable auto flicker mode                |
| <a href="#">set_gain</a>                          | Set gain to register it                         |
| <a href="#">set_auto_flicker_mode</a>             | Set auto flicker mode to enable/disable         |
| <a href="#">get_default_framerate_by_scenario</a> | Get the default framerate by scenario           |
| <a href="#">set_max_framerate_by_scenario</a>     | Set the max framerate by scenario               |
| <a href="#">ihdr_write_shutter_gain</a>           | Write shutter or gain for HDR mode.             |
| <a href="#">set_test_pattern_mode</a>             | Set the test pattern mode for the factory test  |
| <a href="#">return_sensor_id</a>                  | Read sensor id                                  |

#### 6.5.1.1 static kal\_uint32 open

\* FUNCTION

\*       open()

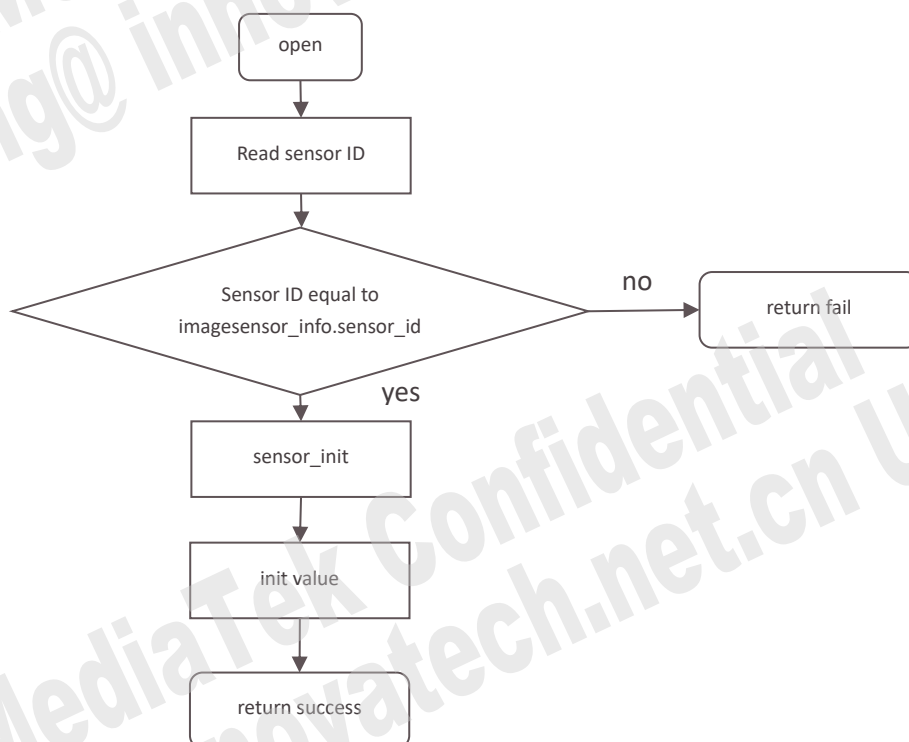
\*

\* DESCRIPTION

\*       This function initializes the registers of CMOS sensor. Check sensor ID, and init value



\*  
 \* PARAMETERS  
 \* None  
 \*  
 \* RETURNS  
 \* None  
 \*  
 \* GLOBALS AFFECTED



#### how to modify:

##### a. check sensor ID

```

sensor_id = return_sensor_id();
static kal_uint32 return_sensor_id(void)
{
    return ((read_cmos_sensor_byte(0x****) << 8) | read_cmos_sensor_byte(0x****));
}
  
```

##### 1. description

Read sensor ID saved in the register

##### 2. how to modify

Fill in the register address saving

##### 3. note

Register address should get FAE/vendor

##### 4. check or warning

If the sensor does not return the correct sensor ID, the possible reason is the register address does not correct.

If I2C communication is not responding, both power input and I2C should be checked.

##### b. call sensor\_init if the check sensor ID OK

```
sensor_init();
```

## Android 15 Camera Legacy Sensor Driver Bring-Up Guide

```
static void sensor_init(void)
{
    LOG_INF("E\n");
    write_cmos_sensor(0x****, 0x****);
    mdelay(3);
    write_cmos_sensor(0x****, 0x****);
    .....
}
```

**1. description**

Fill in the init register setting

**2. How to modify**

init register setting

**3. Note**

Register address and the value should get be obtained from FAE/vendor

**4. Check or warning**

Sensor init usually makes the sensor run, but you can set it to standby mode or directly stream on

**c. Initialize some values in the struct.**

| Member of imgsensor | Default Value                    | Note                               | Need Modify ? |
|---------------------|----------------------------------|------------------------------------|---------------|
| autoflicker_en      | KAL_FALSE                        | related to the autoflicker feature |               |
| sensor_mode         | IMGSENSOR_MODE_INIT              | record sensor mode                 |               |
| shutter             | Depends on your sensor           | record sensor shutter time         | yes           |
| gain                | Depends on your sensor           | record the sensor gain value       | yes           |
| pclk                | imgsensor_info.pre.pclk          | record the current pclk value      |               |
| frame_length        | imgsensor_info.pre.framelength   | record current framelength         |               |
| line_length         | imgsensor_info.pre.linelength    | record current linelength          |               |
| min_frame_length    | imgsensor_info.pre.framelength   | record min framelength             |               |
| dummy_pixel         | 0                                | record current dummy_pixel         |               |
| dummy_line          | 0                                | record current dummy_line          |               |
| hdr_mode            | consult with FAE/vendor          | indicate hdr mode                  | yes           |
| test_pattern        | KAL_FALSE                        | indicate test_pattern state        |               |
| current_fps         | imgsensor_info.pre.max_framerate | record current framerate           |               |

**1. description**

These member of imgsensor is for initial parameters for the sensor

**2. How to modify**

## Android 15 Camera Legacy Sensor Driver Bring-Up Guide

Shutter and gain depend on your sensor; these affect the performance of the first few frames; please fill in the appropriate values according to your registered settings. The `hdr_mode` value depends on your init setting

**3. Note**

Shutter and gain, `hdr_mode` based on your sensor register setting, so you can consult with FAE for filling it out.

**4. check or warning**

If not filled correctly, the initial few frames may appear slightly abnormal. The HDR mode should synchronize with the initialization register settings in this function.

①

**note**

`open()` is an entry function before you can operate this sensor. Register setting should get be from FAE/vendor.

②

**check or warning**

If sensor ID matched fail, might be `return_sensor_id()` doesn't return correct sensor ID. You should check register of sensor ID, another reason is sensor is not power on correctly. `Sensor_init` won't return fail if `sensor_init` fail or work abnormal, so you must confirm `sensor_init()` register correct. Initial value of some struct member in `imgsensor` may be give out, e.g. `imgsensor.shutter`.

**6.5.1.2 static kal\_uint32 get\_info()****\* FUNCTION**

`get_info()`

**\* DESCRIPTION**

This function gets the config info.

**\* PARAMETERS**

enum `MSDK_SCENARIO_ID_ENUM` `scenario_id`,  
scenario case  
`MSDK_SENSOR_INFO_STRUCT` `*sensor_info`  
struct to transfer config info  
`MSDK_SENSOR_CONFIG_STRUCT` `*sensor_config_data`  
address pointer of line numbers in one period of VSYNC

**\* RETURNS**

None

**\* GLOBALS AFFECTED****how to modify:**

Please check the info below

| Member of <code>sensor_info</code> | Default Value                          | Description    |
|------------------------------------|----------------------------------------|----------------|
| <code>SensorClockPolarity</code>   | <code>SENSOR_CLOCK_POLARITY_LOW</code> | clock polarity |

MT8365, MT8371, MT8385, MT8391, MT8766, MT8768, MT8786, MT8788, MT8781, MT8791

## Android 15 Camera Legacy Sensor Driver Bring-Up Guide

|                                         |                                           |                                               |
|-----------------------------------------|-------------------------------------------|-----------------------------------------------|
| SensorClockFallingPolarity              | SENSOR_CLOCK_POLARITY_LOW                 | clock falling polarity                        |
| SensorHsyncPolarity                     | SENSOR_CLOCK_POLARITY_LOW                 | frame Hsync polarity                          |
| SensorVsyncPolarity                     | SENSOR_CLOCK_POLARITY_LOW                 | frame Vsync polarity                          |
| SensorInterruptDelayLines               |                                           | not use                                       |
| SensorResetActiveHigh                   |                                           | not use                                       |
| SensorResetDelayCount                   |                                           | not use                                       |
| SensorInterfaceType                     | imgsensor_info.sensor_interface_type      | Sensor interface type, MIPI or parallel       |
| MIPIsensorType                          | imgsensor_info.mipi_sensor_type           | MIPI protocol type                            |
| SettleDelayMode                         | imgsensor_info.mipi_settle_delay_mode     | Settle delay mode for MIPI, manual or auto    |
| SensorOutputDataFormat                  | imgsensor_info.sensor_output_dataformat   | sensor data format                            |
| CaptureDelayFrame                       | imgsensor_info.cap_delay_frame            | capture delay frame num                       |
| PreviewDelayFrame                       | imgsensor_info.pre_delay_frame            | preview delay frame num                       |
| VideoDelayFrame                         | imgsensor_info.video_delay_frame          | Video delay frame number                      |
| HighSpeedVideoDelayFrame                | imgsensor_info.hs_video_delay_frame       | hs_video delay frame number                   |
| SlimVideoDelayFrame                     | imgsensor_info.slim_video_delay_frame     | slim_video delay frame number                 |
| SensorMasterClockSwitch                 |                                           | not use                                       |
| SensorDrivingCurrent                    | imgsensor_info.isp_driving_current        | isp driving current                           |
| AEShutDelayFrame                        | imgsensor_info.ae_shut_delay_frame        | ae shutter effective delay frame num          |
| AEsensorGainDelayFrame                  | imgsensor_info.ae_sensor_gain_delay_frame | ae gain effective delay frame num             |
| AEISPGainDelayFrame                     | imgsensor_info.ae_ispGain_delay_frame     | AE ISP gains effective delay frame number     |
| IHDR_Support                            | imgsensor_info.ihdr_support               | indicate whether they support ihdr            |
| IHDR_LE_FirstLine                       | imgsensor_info.ihdr_le_firstline          | hdr long exposure and short exposure sequence |
| SensorModeNum                           | imgsensor_info.sensor_mode_num            | supported sensor mode num                     |
| SensorMIPILaneNumber                    | imgsensor_info.mipi_lane_num              | mipi lane num                                 |
| SensorClockFreq                         | imgsensor_info.mclk                       | mclk info                                     |
| SensorClockDividCount                   |                                           | not use                                       |
| SensorClockRisingCount                  | 0                                         | don't care                                    |
| SensorClockFallingCount                 |                                           | not use                                       |
| SensorPixelClockCount                   |                                           | not use                                       |
| SensorDataLatchCount                    |                                           | not use                                       |
| MIPIDataLowPwr2High-SpeedTermDelayCount | 0                                         |                                               |

## Android 15 Camera Legacy Sensor Driver Bring-Up Guide

|                                           |                                                          |                            |
|-------------------------------------------|----------------------------------------------------------|----------------------------|
| MIPICLKLowPwr2High-SpeedTermDelayCount    | 0                                                        |                            |
| SensorWidthSampling                       | 0                                                        |                            |
| SensorHightSampling                       | 0                                                        |                            |
| SensorPacketECCOrder                      | 1                                                        |                            |
| For Each Scenario                         |                                                          |                            |
| SensorGrabStartX                          | imgsensor_info(\$scenario).startx                        | Sensor output image startx |
| SensorGrabStartY                          | imgsensor_info(\$scenario).starty                        | Sensor output image starty |
| MIPIDataLowPwr2High-SpeedSettleDelayCount | imgsensor_info(\$scenario).<br>mipi_data_lp2hs_settle_dc | settle delay num           |

**You should confirm your sensor info right. If there some \*\* means you should base on your sensor fill it .sensor info should learn from spec or consult with vendor/FAE.**

```
.pre = {
    .pclk = **** //pixel clock  pclk = linelength*framelength*fps
    .linelength = **** //length of frame line
    .framelength = **** // length of frame
    .startx = ** // usually 0
    .starty = ** // usually 0
    .grabwindow_width = **** // image width
    .grabwindow_height = **** // image height
    .mipi_data_lp2hs_settle_dc = **** // settle delay count /* unit , ns */
    .max_framerate = **** // frame rate = fps * 10
},
...
.margin = ** /* sensor framelength & shutter margin */
.min_shutter = ** //min shutter */
.min_gain = 64, /*1x gain*/
.max_gain = 960, /*64x gain*/
.min_gain_iso = 100,
.gain_step = 1,
.gain_type = 1, /*to be modify,no gain table for sony*/
.max_frame_length = 0x**, //max frame length
.ae_shut_delay_frame = 0,
.ae_sensor_gain_delay_frame = 0,
.ae_ispGain_delay_frame = 2, /* isp gain delay frame for AE cycle */
.frame_time_delay_frame = 2,
.ihdr_support = *, /* 1, support; 0,not support */
.ihdr_le_firstline = *, /* 1,le first ; 0, se first */
.sensor_mode_num = 7, /* support sensor mode num */
.cap_delay_frame = 2, /* enter capture delay frame num */
.pre_delay_frame = 2, /* enter preview delay frame num */
.video_delay_frame = 2, /* enter video delay frame num */
.hs_video_delay_frame = 2, /* enter high speed video delay frame num */
```

```
.slim_video_delay_frame = 2, /* enter slim video delay frame num */
.custom1_delay_frame = 2, /* add new mode */
.custom2_delay_frame = 2, //usually don't modify the delay frame value
.isp_driving_current = ISP_DRIVING_**MA, /* mclk driving current */
.sensor_interface_type = SENSOR_INTERFACE_TYPE_MIPI,
.mipi_sensor_type = MIPI_OPHY_NCSI2,
.mipi_settle_delay_mode = MIPI_SETTLEDELAY_MANUAL, //you can fill _AUTO either
.sensor_output_dataformat = SENSOR_OUTPUT_FORMAT_**,/
/Refer to kd_imgsensor_define.h and fill the correctly format for your sensor.
.mclk = **, /* mclk value, suggest 24 or 26 for 24Mhz or 26Mhz */
.mipi_lane_num = SENSOR_MIPI_**_LANE, /* mipi lane num */
.i2c_addr_table = {0x**, 0x**, 0xff}, // i2c available address , 0xff is necessary
```

①

**note**

This function does not need modification because most parameters can be filled in `imgsensor_info`, while other parameters can be kept at their default values.

②

**check or warning**

Please fill in `imgsensor_info` that refers to the above table, usually need to modify the red font part. Please double confirm it with the spec or FAE/vendor.

### 6.5.1.3 static kal\_uint32 get\_resolution

```
* FUNCTION
*   get_resolution()
*
* DESCRIPTION
*   get sensor resolution for different scenario
*
* PARAMETERS
*   MSDK_SENSOR_RESOLUTION_INFO_STRUCT(*sensor_resolution)
*   transfer resolution info to user space
*
* RETURNS
*   None
*
* GLOBALS AFFECTED
```

**how to modify:**

No need for modifications; fill in the grabwindow info in `imgsensor_info`,

①

**note**

Different scenarios maybe have different resolutions, please fill in it and sync with the `winsize_info` struct.

②

**check or warning**

### 6.5.1.4 static kal\_uint32 feature\_control

```
* FUNCTION
*   feature_control()
```



```

*
* DESCRIPTION
*     This function implement sensor feature from hal
*
* PARAMETERS
*     MSDK_SENSOR_FEATURE_ENUM feature_id
*     cmd of MSDK_SENSOR_FEATURE_ENUM means different feature
*
*     UINT8 *feature_para
*     parameters transfer data between kernel & hal
*
*     UINT32 *feature_para_len
*     feature parameter length
*
* RETURNS
*     None
*
* GLOBALS AFFECTED

```

**how to modify:**

You do not need to change feature\_control; you just need to modify the function it calls. I will introduce cmd in feature\_control later.

```
switch (feature_id) {
```

**a) case SENSOR\_FEATURE\_GET\_PERIOD:**

```

    *feature_return_para_16++ = imgsensor.line_length;
    *feature_return_para_16 = imgsensor.frame_length;
    *feature_para_len = 4;
    break;

```

Get the line\_length and frame\_length from the imgsensor struct. Calculate the linetime with the pclk returned to the upper layer to convert the shutter corresponding to the exposure time of the ae plinetable

**b) case SENSOR\_FEATURE\_GET\_PIXEL\_CLOCK\_FREQ:**

```

    *feature_return_para_32 = imgsensor.pclk;
    *feature_para_len = 4;
    break;

```

get pclk, no need to modify this case.but you need confirm line\_length, frame\_length, pclk filled correctly. **Example:**

```

    .pre = {
        .pclk = 39000000,    // please fill it base on your sensor
        .linelength = 1040  // please fill it base on your sensor
        .framelength = 1240, // please fill it base on your sensor
        .max_framerate = 300, // please fill it base on your sensor
    },

```

Pclk = linelength \* frame\_length \* fps // the PCLK can be slightly larger than the calculated value.Fps = max\_framerate / 10

**c) case SENSOR\_FEATURE\_SET\_ESHUTTER:**

```
set_shutter((UINT16) *feature_data);
```



```

break;
i. description
    * FUNCTION
    *set_shutter
    *
    * DESCRIPTION
    * This function set shutter value to register, before set will calculator
    * framelength and linelength of imgsensor
    *
    * PARAMETERS
    * kal_uint16 shutter
    * shutter value
    *
    * RETURNS
    * None
    *
    * GLOBALS AFFECTED

```

ii. **how to modify**

```

static void set_shutter(kal_uint16 shutter)
{
    ...
    if (shutter > imgsensor.min_frame_length - imgsensor_info.margin)
        /* Extend frame length */
        write_cmos_sensor(0x**01, 0x0*);
        write_cmos_sensor(0x**02, imgsensor.frame_length >> 8);
        write_cmos_sensor(0x**03, imgsensor.frame_length & 0xFF);
        write_cmos_sensor(0x**01, 0x0#);

        /* Update Shutter */
        write_cmos_sensor(0x**01, 0x0*);
        write_cmos_sensor(0x**04, 0x0*); /* Enable auto extend */
        write_cmos_sensor(0x**05, (shutter >> 8) & 0xFF);
        write_cmos_sensor(0x**06, shutter & 0xFF);
        write_cmos_sensor(0x**01, 0x0*);
    ...
}/* set_shutter */

```

0x\*\*01 & 0x\*\*04 is determine auto extend enable/disable if your sensor support auto extend. You should ask FAE for details. You can ignore it if don't support.

0x\*\*02 & 0x\*\*03 is the register save the frame\_length

0x\*\*05 & 0x\*\*06 is the register save the shutter

iii. **note**

register address 0x\*\*01 0x\*\*02 0x\*\*03 0x\*\*04 0x\*\*05 0x\*\*06 is dummy register address, you should check the spec or ask for FAE for it . 0x0\* or 0x0# is value to set when auto extend, to enable/disable auto extend

iv. **check or warning**

## Android 15 Camera Legacy Sensor Driver Bring-Up Guide

In the `set_shutter` function, you need to modify the register to save the shutter and `imgsensor_framelength`. If not done correctly, the image of the preview or video and the picture might be abnormal in brightness. If sensor support auto extend frame\_length, you should consult with FAE register setting sequence.

d) **case SENSOR\_FEATURE\_SET\_GAIN:**

[set\\_gain](#)((UINT16) \*feature\_data);

break;

i. **description**

\*set\_gain

\*

\* DESCRIPTION

\* This function is to set global gain to sensor.

\*

\* PARAMETERS

\* kal\_uint16 gain

\* sensor gain to be set.

\*

\* RETURNS

\* the actually gain set to sensor.

\*

\* GLOBALS AFFECTED

ii. **how to modify**

```
static kal_uint16 set_gain(kal_uint16 gain)
```

```
{
```

```
.....
```

```
reg_gain = gain2reg(gain);
```

```
.....
```

```
write_cmos_sensor(0x**01, 0x0#);
```

```
write_cmos_sensor(0x**02, (reg_gain >> 8) & 0xFF);
```

```
write_cmos_sensor(0x**03, reg_gain & 0xFF);
```

```
write_cmos_sensor(0x**01, 0x0*);
```

```
return gain;
```

```
}
```

0x\*\*01 determine enable/disable auto extend framelength if your sensor support auto extend. You can ignore it if don't support.

0x\*\*02 & 0x\*\*03 save reg\_gain transferred by [gain2reg](#).

iii. **note**

For different camera sensors, the gain value transfer may be different. It might use a gain table or setting in the register directly.

iv. **check or warning**

Setting the gain is crucial for sensor performance as there are multiple methods available for adjusting it. If the current approach to setting the gain does not impact the camera sensor initialization, adjustments can be made during CCT execution. When the linearity of CCT gain aligns with specifications, it indicates that there are no issues with how the gain is set. Otherwise, modifications should be discussed with FAE/vendor.

```
static kal_uint16 gain2reg(const kal_uint16 gain)
```

```
{
```

```

kal_uint8 i;

for (i = 0; i < IMX***MIPI_MaxGainIndex; i++) {
    if (gain <= IMX***MIPI_sensorGainMapping[i][0])
        break;
}
...
return IMX***MIPI_sensorGainMapping[i][1];
}

```

The gain table is used to solve the linear relationship between the gain value and the register setting, or it cannot be directly calculated. gain2reg This function can map the input gain value to the value that needs to be written to the register. If some camera sensors can obtain the register value corresponding to gain through calculation, we can either obtain reg gain through calculation or map through a gain table.

**e) case SENSOR\_FEATURE\_SET\_AUTO\_FLICKER\_MODE:**

```

set_auto_flicker_mode( (BOOL) * feature_data_16, *(feature_data_16 + 1));
break;

```

set\_auto\_flicker\_mode called when auto flicker needed, it can avoid 15 fps and 30fps, these fps will cause algorithm check fail when some scenario.

```

static kal_uint32 set_auto_flicker_mode(kal_bool enable, UINT16 framerate)
{

```

```

    ...
    spin_lock(&imgsensor_drv_lock);
    if (enable) /* enable auto flicker */
        imgsensor.autoflicker_en = KAL_TRUE;
    else /* Cancel Auto flick */
        imgsensor.autoflicker_en = KAL_FALSE;
    spin_unlock(&imgsensor_drv_lock);
    return ERROR_NONE;
}

```

**f) case SENSOR\_FEATURE\_SET\_MAX\_FRAME\_RATE\_BY\_SCENARIO:**

```

set_max_framerate_by_scenario(
(enum MSDK_SCENARIO_ID_ENUM) *feature_data, *(feature_data + 1));
break;

```

**case SENSOR\_FEATURE\_GET\_DEFAULT\_FRAME\_RATE\_BY\_SCENARIO:**

```

get_default_framerate_by_scenario(
(enum MSDK_SCENARIO_ID_ENUM) *feature_data,
(MUINT32 *) (uintptr_t) (*(feature_data + 1)));
break;

```

The maximum framerate can be set or retrieved based on the scenario; there is no requirement to modify these two functions.

**g) case SENSOR\_FEATURE\_SET\_TEST\_PATTERN:**

```

set_test_pattern_mode((BOOL) * feature_data);
break;

```

## Android 15 Camera Legacy Sensor Driver Bring-Up Guide

Camera auto test in Factory mode will let the sensor output the test pattern through this interface for CRC verification. The test pattern setting must ensure that the same bit of data output by the sensor is completely consistent. Please provide the setting from the vendor.

i. **Description**

## FUNCTION

\* set\_test\_pattern\_mode

\*

## \* DESCRIPTION

\* this function enable/disable test pattern mode

\*

## \* PARAMETERS

\*

kal\_bool enable

\* indicate test pattern mode enable/disable

\*

## \* RETURNS

\* None

\*

## \* GLOBALS AFFECTED

ii. **How to modify**

```
static kal_uint32 set_test_pattern_mode(kal_bool enable)
```

```
{
```

```
    if (enable)
```

```
        write_cmos_sensor(0x**01, 0x**);
```

```
    else
```

```
        write_cmos_sensor(0x**01, 0x**);
```

```
    .....
```

```
}
```

iii. **Note**

Need to confirm with the vendor/FAE how to set the test pattern enable/disable

iv. **Check or warning**

Check the CRC checksum in factory mode.

h) **case SENSOR\_FEATURE\_GET\_CROP\_INFO:**

```
...
```

```
wininfo =
```

```
(struct SENSOR_WINSIZE_INFO_STRUCT *) (uintptr_t) (*(feature_data + 1));
```

```
switch (*feature_data_32) {
```

```
.....
```

```
case MSDK_SCENARIO_ID_CAMERA_PREVIEW:
```

```
default:
```

```
    memcpy((void *)wininfo,
```

```
        (void *)&imgsensor_winsize_info[0],
```

```
        sizeof(struct SENSOR_WINSIZE_INFO_STRUCT));
```

```
    break;
```

```
}
```

```
break;
```

i. **description**

Get crop info from driver structure `imgsensor_winsize_info`  
for how to fill in `imgsensor_winsize_info`

## ii. how to modify

Confirm your `imgsensor_winsize_info` is correctly set.

**Full size:** original full size in pixels

**Crop1:** Get the crop window size and offset before resizing

Resize: width and height after resizing in the sensor

**Crop2:** sensor crop again after resizing (not all sensors needed)

**ISP Crop:** crop in an ISP

```
static struct SENSOR_WINSIZE_INFO_STRUCT imgsensor_winsize_info[*] = {
    {full_w, full_h, x0_offset, y0_offset, w0_size, h0_size, scale_w, scale_h, x1_offset,
    y1_offset, w1_size, h1_size, x2_tg_offset, y2_tg_offset, w2_tg_size, h2_tg_size}, //preview
    {full_w, full_h, x0_offset, y0_offset, w0_size, h0_size, scale_w, scale_h, x1_offset,
    y1_offset, w1_size, h1_size, x2_tg_offset, y2_tg_offset, w2_tg_size, h2_tg_size}, //capture
    . . . . .
}
```

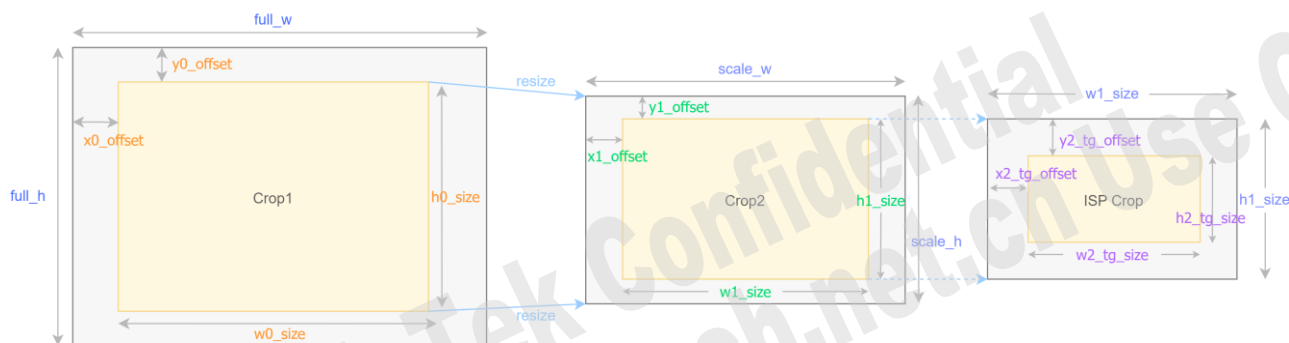
## Sensor Crop Information

### Sensor output

- crop1 -> resize (binning) -> crop2 then ISP crop information (TG crop)

### Backward to full resolution

- The Sensor driver provides the crop region:
  - Full size w/h (capture raw size): `full_w`, `full_h`
  - Crop1 information: `x0_offset`, `y0_offset`, `w0_size`, `h0_size`
  - Resize output width/height: `scale_w`, `scale_h`
  - Crop2 information: `x1_offset`, `y1_offset`, `w1_size`, `h1_size`
- ISP Crop (TG) information
  - TG windows w/h: `w2_tg_size`, `h2_tg_size`
  - TG offset: `x2_tg_offset`, `y2_tg_offset`



## iii. Note

Modification is unnecessary for this case. It should be noted that

**iv. check or warning**

For LSC use, the information needs to be filled in correctly, otherwise it may affect the result so that the preview may not be possible, or the LSC effect is not good.

**i) case SENSOR\_FEATURE\_SET\_IHDR\_SHUTTER\_GAIN:**

```
ihdr_write_shutter_gain_((UINT16) *feature_data,
                          (UINT16) *(feature_data + 1),
                          (UINT16) *(feature_data + 2));

break;
```

**i. description**

\* FUNCTION

\* ihdr\_write\_shutter\_gain

\*

\* DESCRIPTION

\* This function is to set global gain for HDR sensor mode.

\*

\* PARAMETERS

\* kal\_uint16 le, kal\_uint16 se, kal\_uint16 gain

\* le : long exposure for hdr mode

\* se:short exposure for hdr mode

\* gain: gain for hdr mode

\*

\* RETURNS

\* the actual gain set to sensor.

\*

\* NONE

**ii. how to modify**

```
static void ihdr_write_shutter_gain(kal_uint16 le, kal_uint16 se, kal_uint16 gain)
```

```
{
```

```
.....
```

```
} else {
```

```
    write_cmos_sensor(0x**01, 0x01);
```

```
    write_cmos_sensor(0x**02, imgsensor.frame_length >> 8);
```

```
    write_cmos_sensor(0x**03, imgsensor.frame_length & 0xFF);
```

```
    write_cmos_sensor(0x**01, 0x00);
```

```
}
```

```
write_cmos_sensor(0x**01, 0x01);
```

```
write_cmos_sensor(0x**04, 0x01); /* Enable auto extend */
```

```
/* Long exposure */
```

```
write_cmos_sensor(0x**05, (le >> 8) & 0xFF);
```

```
write_cmos_sensor(0x**06, le & 0xFF);
```

```
/* Short exposure */
```

```
write_cmos_sensor(0x**07, (se >> 8) & 0xFF);
```

```
write_cmos_sensor(0x**08, se & 0xFF);
```

```
reg_gain = gain2reg(gain);
```



```

spin_lock(&imgsensor_drv_lock);
imgsensor.gain = reg_gain;
spin_unlock(&imgsensor_drv_lock);
/* Global analog Gain for Long expo */
write_cmos_sensor(0x**09, (reg_gain >> 8) & 0xFF);
write_cmos_sensor(0x**10, reg_gain & 0xFF);
/* Global analog Gain for Short expo */
/* write_cmos_sensor(0x**11, (reg_gain>>8)& 0xFF); */
/* write_cmos_sensor(0x**12, reg_gain & 0xFF); */
write_cmos_sensor(0x**01, 0x00);
... ..
}

```

0x\*\*01 & 0x\*\*04 is determine auto extend enable/disable if your sensor support auto extend. you should ask FAE for details. You can ignore it if don't support.

0x\*\*02 & 0x\*\*03 is the register save the frame\_length

0x\*\*05 & 0x\*\*06 is the register save long exposure value

0x\*\*07 & 0x\*\*08 is the register save short exposure

0x\*\*09 & 0x\*\*10 is the register save the analog gain for long exposure

0x\*\*11 & 0x\*\*12 is the register save the analog gain for short exposure

All these registers should be confirmed with the FAE/vendor to determine if the camera module is an iHDR module.

### iii. **Note**

In comparison to set\_shutter, this function is utilized in iHDR mode. The standard camera module has a single exposure value, whereas iHDR mode necessitates both long and short exposures.

### iv. **check or warning**

in iHDR\_write\_shutter\_gain function, you need to modify the register save the shutter, ie, se, gain, imgsensor\_framelength. If not do correctly, image of preview or video and picture might be abnormal in brightness for iHDR mode. If sensor support auto extend frame\_length, should consult with FAE register setting sequence.

### j) **case SENSOR\_FEATURE\_SET\_SHUTTER\_FRAME\_TIME:**

```

set_shutter_frame_length(
    (UINT16) *feature_data, (UINT16) *(feature_data + 1));
break;

```

#### i. **description**

Compared with set\_shutter, this function uses dummy\_line for calculation.

#### ii. **how to modify**

refer to [set\\_shutter](#).

#### iii. **note**

Utilized for CCU frame synchronization.

#### iv. **check or warning**

Under normal circumstances, we don't usually use this function. If you do use the ccu frame sync mechanism, but it does not work, please check whether the auto extend function is enabled. This requires the vendor/FAE to help confirm.

### k) **case SENSOR\_FEATURE\_SET\_STREAMING\_SUSPEND:**

- streaming\_control(KAL\_FALSE);



- break;

- **case SENSOR\_FEATURE\_SET\_STREAMING\_RESUME:**

.....

```
if (*feature_data != 0)
    set_shutter(*feature_data);
streaming_control(KAL_TRUE);
break;
static void check_streamoff(void)
{
    ...
    mdelay(3);
    for (i = 0; i < timeout; i++) {
        if (read_cmos_sensor_8(0x**02) != 0xFF) //check streaming state
            mdelay(5);
        else
            break;
    }
    ...
}
static kal_uint32 streaming_control(kal_bool enable)
{
    if (enable) {
        /* MIPI// */
        write_cmos_sensor(0x**01, 0x##); /*Stream on */
    } else {
        /* MIPI// */
        write_cmos_sensor(0x**01, 0x%%); /*Stream off */
    }
    check\_streamoff\(\);
    /*if no register record stream state,do not call this,use mdelay(10) */
    //mdelay(10);
    return ERROR_NONE;
}
```

**i. description**

streaming\_control is used for streaming on/off control; it is necessary for ISP 3.0 and later versions.

**ii. how to modify**

0x\*\*01 means register control streaming state.0x\*\*02 record whether streaming of one frame is end, it could be provided by FAE/vendor. IF camera sensor does not support check whether streaming of one frame is end, you should take some delay after setting on/off, this value should preferably exceed one frame time.you can calculator it by 1000/current\_fps .

**iii. note**

This function is useful for the camera sensor timing issue.

**iv. check or warning**

If the sensor does not support this function or is not implemented correctly, there may be some trouble after sensor bring up. The problems caused by timing are both difficult to detect and difficult to find. To make the subsequent stress test less troublesome, please implement this function correctly.

**l) case SENSOR\_FEATURE\_GET\_MIPI\_PIXEL\_RATE:**

```
{
    kal_uint32 rate;
    switch (*feature_data) {
        ... ..
        case MSDK_SCENARIO_ID_CAMERA_PREVIEW:
            default:
                rate = imgsensor_info.pre.mipi_pixel_rate;
                break;
    }
    *(MUINT32 *) (uintptr_t) (*(feature_data + 1)) = rate;
}
```

break;

This case is used for dynamically adjusting buffer size before streaming. Consult with the FAE/vendor for MIPI data rate for each scenario.

**m) set\_dummy**

**i. description**

```
* FUNCTION
*
*     set_dummy
*
* DESCRIPTION
*
*     This function update framelength , linelength or vb ,gain, shutter.
*
* PARAMETERS
*
*     NONE
*
* RETURNS
*
*     None
*
* GLOBALS AFFECTED
```

**ii. how to modify**

```
static void set_dummy(void)
```

```
{
    write_cmos_sensor(0x**01, 0x01);
    write_cmos_sensor(0x**02, imgsensor.frame_length >> 8);
    write_cmos_sensor(0x**03, imgsensor.frame_length & 0xFF);
    write_cmos_sensor(0x**04, imgsensor.line_length >> 8);
    write_cmos_sensor(0x**05, imgsensor.line_length & 0xFF);
    write_cmos_sensor(0x**01, 0x00);
}
```

0x\*\*01 is register determine auto extend ,you can delete it if your camera sensor does not support this feature.0x\*\*02 0x\*\*03 save the frame\_length , 0x\*\*04 0x\*\*05 save the

line\_length, you need to consult with FAE/vendor for these register address or learn it from spec.

iii. **note**

Frame\_length and line\_length are always set in set\_dummy. In some cases, they need to be implemented by setting vb, gain, shutter, and other parameters. Consult with FAE/vendor to determine which parameters should be set at set\_dummy. There are many issues with scene changing.

iv. **check or warning**

set\_dummy is always used after setting gain, shutter, and framelength. If you do not implement correctly or not call correctly, there will be

n) case SENSOR\_FEATURE\_GET\_GAIN\_RANGE\_BY\_SCEARIO:

```
*(feature_data + 1) = imgsensor_info.min_gain;
```

```
*(feature_data + 2) = imgsensor_info.max_gain;
```

```
break;
```

o) case SENSOR\_FEATURE\_GET\_BASE\_GAIN\_ISO\_AND\_STEP:

```
*(feature_data + 0) = imgsensor_info.min_gain_iso;
```

```
*(feature_data + 1) = imgsensor_info.gain_step;
```

```
*(feature_data + 2) = imgsensor_info.gain_type;
```

```
break;
```

p) case SENSOR\_FEATURE\_GET\_MIN\_SHUTTER\_BY\_SCENARIO:

```
*(feature_data + 1) = imgsensor_info.min_shutter;
```

```
Switch(*feature_data) {
```

```
case MSDK_SCENARIO_ID_CAMERA_PREVIEW:
```

```
case MSDK_SCENARIO_ID_CAMERA_CAPTURE_JPEG:
```

```
case MSDK_SCENARIO_ID_VIDEO_PREVIEW:
```

```
case MSDK_SCENARIO_ID_SLIM_PREVIEW:
```

```
case MSDK_SCENARIO_ID_HIGH_SPEED_VIDEO:
```

```
*(feature_data + 2) = 2;
```

```
break;
```

```
case MSDK_SCENARIO_ID_CUSTOM3:
```

```
default:
```

```
*(feature_data + 2) = 1;
```

```
break;
```

```
}
```

```
break;
```

i. **check or warning**

min\_gain,max\_gain,min\_gain\_iso,gain\_step,gain\_type is kal\_uint32 variable should be define in sensor driver header file which like xxx\_mipiraw\_sensor.h,and cmd contains

SENSOR\_FEATURE\_GET\_GAIN\_RANGE\_BY\_SCEARIO

SENSOR\_FEATURE\_GET\_BASE\_GAIN\_ISO\_AND\_STEP

SENSOR\_FEATURE\_GET\_MIN\_SHUTTER\_BY\_SCENARIO

**must** be added, this will cause AE to be unable to obtain information to calculate undesired results.

①

**note**

Here are just some common feature control cases. If you want to add a new cmd, you need to check whether there is a feature ID item that can be used in the kd\_imgsensor\_define.h file. If you need to add a new featureID,imgsensor/inc/kd\_imgsensor\_define.h and device /mediatek/common/kd\_imgsensor\_define.h should add new feature IDs, and keep the same order.

②

**check or warning**

When we suspect that some feature is not implemented correctly, we need to give priority to whether the implementation of this feature is called. You can add some debug information below the case statement.

### 6.5.1.5 static kal\_uint32 control

```
* FUNCTION
*
*      control
*
* DESCRIPTION
*
*      control sensor mode
*
* PARAMETERS
*
*      enum MSDK_SCENARIO_ID_ENUM scenario_id
*
*      scenario id
*
*      MSDK_SENSOR_EXPOSURE_WINDOW_STRUCT *image_window
*
*      address pointer of pixel numbers in one period of HSYNC
*
*      MSDK_SENSOR_CONFIG_STRUCT *sensor_config_data
*
*      address pointer of line numbers in one period of VSYNC
*
* RETURNS
*
*      None
*
* GLOBALS AFFECTED
```

#### ① how to modify

No need to modify it

```
static kal_uint32 control(
    enum MSDK_SCENARIO_ID_ENUM scenario_id,
    MSDK_SENSOR_EXPOSURE_WINDOW_STRUCT *image_window,
    MSDK_SENSOR_CONFIG_STRUCT *sensor_config_data)
{
    ... ..
    switch (scenario_id) {
        ... ..
        default:
            cam_pr_debug("Error ScenarioId setting");
            preview(image_window, sensor_config_data);
            return ERROR_INVALID_SCENARIO_ID;
    }

    return ERROR_NONE;
}

/* control() */
```

#### ②

**note**

#### ③

**check or warning**

### 6.5.1.6 static kal\_uint32 close

\* FUNCTION

\* close

\*

\* DESCRIPTION

\* do uninit if necessary.

\*

\* PARAMETERS

\* None

\*

\* RETURNS

\* None

\*

\* GLOBALS AFFECTED

#### ① how to modify

No need if your sensor does not need to uninit.

#### ② Note

Usually no need to implement, most camera sensor don't need uninit sensor.

#### ③ check or warning

In most cases, we use power off instead of close().

## 7 User Space Porting

### 7.1 Modify ProjectConfig.mk

#### Description:

In Chapter 6.2 describes the compilation options in the kernel space. Here are the compilation options at the user space.

#### Path:

```
device/mediateksample/{$Project}/ProjectConfig.mk
```

#### Example:

```
device/mediateksample/aiot8863p2_64_bsp/ProjectConfig.mk
```

#### Add ov13855\_mipi\_raw

```
CUSTOM_HAL_IMGSENSOR = imx214_mipi_raw imx256_mipi_raw ov13855_mipi_raw  
CUSTOM_KERNEL_IMGSENSOR = imx214_mipi_raw imx256_mipi_raw ov13855_mipi_raw
```

**Note:** Add all the sensors needed to be built-in, space should be inserted between every two sensors, after them are compatible sensors.

### 7.2 Modify CameraConfig.mk

#### Description:

In Chapter 6.2 describes the compilation options in the kernel space. Here are the compilation options at the user space.

#### Path:

```
device/mediatek/{$SW_CODE_PATH}/CameraConfig.mk
```

#### Example:

```
device/mediatek/mt6835/CameraConfig.mk
```

#### Add ov13855\_mipi\_raw

```
CUSTOM_HAL_IMGSENSOR = imx214_mipi_raw imx256_mipi_raw ov13855_mipi_raw  
CUSTOM_KERNEL_IMGSENSOR = imx214_mipi_raw imx256_mipi_raw ov13855_mipi_raw
```

**Note:** Add all the sensors needed to be built-in, space should be inserted between every two sensors, after them are compatible sensors.

### 7.3 Modify device-camera.mk

#### Description:

Here is used to configure the sensor list for IQ tuning binary files.

#### Path:

```
device/mediatek/{$SW_CODE_PATH}/device-camera.mk
```



**Example:**

```
device/mediatek/mt6835/device-camera.mk
```

**Add ov13855\_mipi\_raw**

```
CUSTOM_HAL_IMGSENSOR = imx214_mipi_raw imx256_mipi_raw ov13855_mipi_raw
```

**Note:** Add all the sensors needed to be built-in, space should be inserted between every two sensors, after them are compatible sensors.

## 7.4 Add a New Sensor to mtkcamvars.go

**Description:**

The corresponding sensors should be added.

`{SensorName}_IdxMgr`,  
`{SensorName}_tuning`,  
`ISP_mapping_{SensorName}`,  
`ISP_param_{SensorName}`, and  
`libCamera_{SensorName}_{TuningScenario}`  
to `mtkcamModuleList`.

**Path:**

```
vendor/mediatek/proprietary/scripts/soong/mtkcam/mtkcamvars.go
```

**Example:**

```
14
15 func MtkcamModuleList() []string {
16     var mtkcamModuleList = []string {
17         "imx214_mipi_raw_IdxMgr",
18         "imx214_mipi_raw_tuning",
19         "ISP_mapping_imx214_mipi_raw",
20         "ISP_param_imx214_mipi_raw",
21         "libCamera_imx214mipiraw_Face_Capture",
22         "libCamera_imx214mipiraw_Face_Capture_4cell",
23         ...
24
```

## 7.5 Config Hardware Related (mclk CSI)

**Description:**

Configure the HW-related settings at the user layer; set the GPIO port of mclk on the GPIO port. mclk is provided by the ISP, and here you can configure the source of clk.

**Path:**

## Android 15 Camera Legacy Sensor Driver Bring-Up Guide

```
vendor/mediatek/proprietary/custom/{$Project|$Platform|common}/hal/imgsensor_src/
cfg_setting_table.cpp
```

**Note:** If `cfg_setting_table.cpp` exists in the `{$Project}` path, please modify it there. If it cannot be found in the `{$Project}` path, then modify it in the `{$Platform}` path. If it cannot be found in either location, then modify it in the `common` path.

```
static CUSTOM_CFG gCustomCfg[] = {
{
    .sensorIdx    = IMSENSOR_SENSOR_IDX_MAIN,
    .mclk         = CUSTOM_CFG_MCLK_2,
    .port         = CUSTOM_CFG_CSI_PORT_1,
    .dir          = CUSTOM_CFG_DIR_REAR,
    .bitOrder     = CUSTOM_CFG_BITORDER_9_2,
    .orientation  = 90,
    .horizontalFov = 67,
    .verticalFov  = 49
},

typedef enum {
    CUSTOM_CFG_MCLK_1 = 0x0,    //mclk1
    CUSTOM_CFG_MCLK_2,    //mclk2
    CUSTOM_CFG_MCLK_3,    //mclk3
    CUSTOM_CFG_MCLK_4,
    CUSTOM_CFG_MCLK_5,
    CUSTOM_CFG_MCLK_MAX_NUM,
    CUSTOM_CFG_MCLK_NONE
} CUSTOM_CFG_MCLK;
```

The above options configure the source of mclk and the number of the port. As you can see above, the current rear sensor MIPI uses 4-lane to transmit data, and the connection is port\_0.

## 7.6 Add New Sensor ID and Name

### Description:

Add the sensor ID and name in the user space.

### Path:

```
device/mediatek/common/kernel-headers/kd_imgsensor.h
```

### Config sensor ID

```
#define OV13855_SENSOR_ID          0xD855
```

### Define the sensor device driver name

```
#define SENSOR_DRVNAME_OV13855_MIPI_RAW    "ov13855mipiraw"
```

**Note:** Keep consistent with the user space.

## 7.7 Add New Sensor to Sensor List

### Description:

Also configure the callback declaration function the user space

### Path:

```
vendor/mediatek/proprietary/custom/${Project}|$Platform|common}/hal/imgsensor_src/  
sensorlist.cpp
```

**Note:** If sensorlist.cpp exists in the **\${Project}** path, please modify it there. If it cannot be found in the **\${Project}** path, then modify it in the **\${Platform}** path. If it cannot be found in either location, then modify it in the **common** path.

### Example:

Add new sensor in hal sensorlist

```
#if defined(OV13855_MIPI_RAW)  
RAW_INFO_M(OV13855_SENSOR_ID, DEFAULT_MODULE_INDEX, DEFAULT_MODULE_ID, \  
SENSOR_DRVNAME_OV13855_MIPI_RAW, NULL),
```

**Note:** sensorlist[] in sensorlist.cpp and imgsensor\_sensor\_list.c, the items they contained, must have the same sequence order, or else, will get a wrong sensor ID when searching sensor.

## 7.8 Add Tuning/Metadata Files

### Description:

This corresponds to the file preparation in Chapter 5, and here is a detailed description of the specific method and path of adding files.

- (1) metadata (vendor/mediatek/proprietary/custom/common/hal/imgsensor\_metadata/sensor/<sensor>)
- (2) metadata (vendor/mediatek/proprietary/custom/{**\$Platform**}/hal/imgsensor\_metadata/<sensor>)
- (3) tuning (vendor/mediatek/proprietary/custom/{**\$Platform**}/hal/imgsensor/ver1/<sensor>)

### Note:

- [1] If sensor driver in kernel (please refer to MTK MP-sensor architecture) is ready, please put into the kernel driver path, otherwise need vendor to provide.
- [2] If hal files not ready, please rename/copy one folder named by MTK MP-sensor, which can be known by asking CPM or driver RD, tuning and metadata belong to HAL file.

Sensor code (metadata/ tuning file...) and some customer setting config folder tree in user space as below:

```

└─ vendor
  └─ mediatek
    └─ proprietary
      └─ custom
        ├── common
        │   ├── hal
        │   │   └─ imgsensor_metadata
        │   │       └─ sensor
        │   │           └─ xxxx_mipi_raw
        │   │               └─ config_static_metadata.sensor.xxxxmipiraw.h
        │   └─ {$Platform}
        │       ├── hal
        │       │   ├── imgsensor_metadata
        │       │   │   ├── config_static_metadata.xxx.xxxxmipiraw.h
        │       │   │   ├── config_static_metadata_xxxx.h
        │       │   │   └─ ...
        │       └─ imgsensor
        │           └─ ver1
        │               └─ {SENSOR_DRV_NAME}
        │                   ├── camera_tuning_xxxx
        │                   ├── camera_xxxx_para_xxx
        │                   └─ ...

```

### 7.8.1 How to Prepare for Your Tuning File

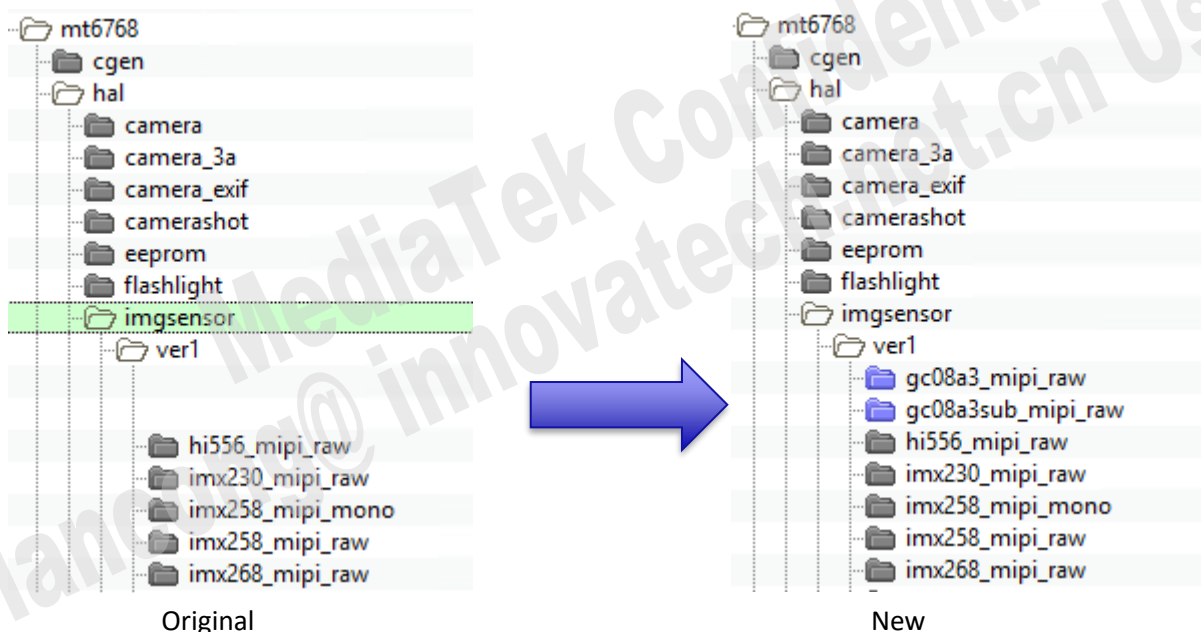
Tuning file:

Copy an existing sensor tuning file that can work on our platform to the imgsensor folder and rename it.

e.g.: copy imx230\_mipi\_raw and rename our sensor tuning file gc08a3\_mipi\_raw && gc08a3sub\_mipi\_raw.

MT8365, MT8371, MT8385, MT8391, MT8766, MT8768, MT8786, MT8788, MT8781, MT8791

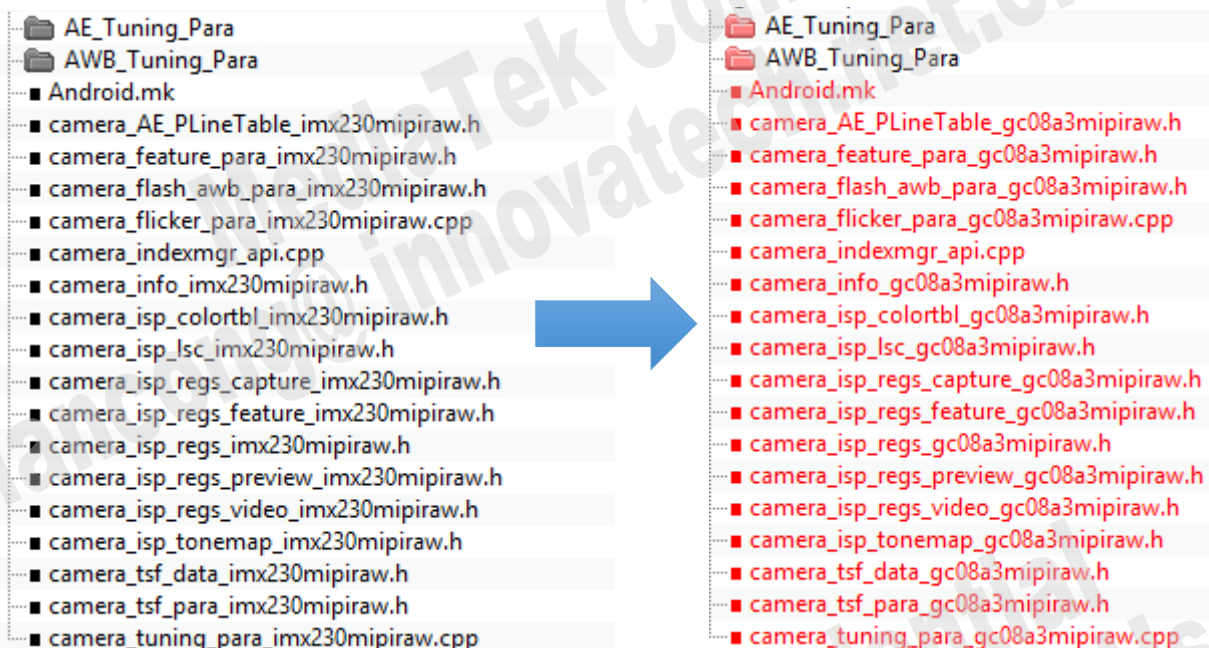
## Android 15 Camera Legacy Sensor Driver Bring-Up Guide



Enter tuning file folder, copy which could work on our platform, for instance imx230\_mipi\_raw and rename our sensor tuning name gc08a3\_mipi\_raw && gc08a3sub\_mipi\_raw.

As can be seen from the picture above, gc08a3\_mipi\_raw && gc08a3sub\_mipi\_raw are our new sensor tuning file folder. Then Change all the new tuning file to the name of the new Sensor.

e.g.: Entering the gc08a3\_mipi\_raw folder, we can see the following below:



Replace all files which name containing "imx230" with "gc08a3", then open each file do the same.

e.g.: open camera\_indexmgr\_api.cpp, replace keywords "imx230" with "gc08a3".

## Android 15 Camera Legacy Sensor Driver Bring-Up Guide

```
#include <utils/Log.h>
#include <fcntl.h>
#include <math.h>
#include <cutils/log.h>
#include <string.h>

#include "camera_custom_nvram.h"
#include "camera_custom_sensor.h"
#include "image_sensor.h"
#include "kd_imgsensor_define.h"

//camera_info_header
#include "camera_info_imx230mipiraw.h"
```



```
#include <utils/Log.h>
#include <fcntl.h>
#include <math.h>
#include <cutils/log.h>
#include <string.h>

#include "camera_custom_nvram.h"
#include "camera_custom_sensor.h"
#include "image_sensor.h"
#include "kd_imgsensor_define.h"

//camera_info_header
#include "camera_info_gc08a3mipiraw.h"
```

The picture above is a part of the same operation in each file content.

The tuning team needs to complete the specific parameters of each tuning file related to the sensor.

The same operation is applied for metadata files.

## 7.8.2 How to Prepare for Your Metadata File

| metadata tag                                                  | description                     |
|---------------------------------------------------------------|---------------------------------|
| <a href="#">MTK_SCALER_AVAILABLE_STREAM_CONFIGURATIONS</a>    | available stream configurations |
| <a href="#">MTK_SENSOR_INFO_OUTPUT_REGION_ON_ACTIVE_ARRAY</a> | sensor output useful array size |
| <a href="#">MTK_SENSOR_INFO_REAL_OUTPUT_SIZE</a>              | Sensor output, full size        |
| <a href="#">MTK_SENSOR_INFO_ORIENTATION</a>                   | Sensor orientation for API 2    |
| <a href="#">MTK_SENSOR_INFO_WANTED_ORIENTATION</a>            | Sensor orientation for API1     |
| <a href="#">MTK_SENSOR_INFO_FACING</a>                        | sensor device facing            |
| <a href="#">MTK_SENSOR_INFO_PIXEL_ARRAY_SIZE</a>              | sensor max size                 |

**Attention:** Here is to configure some metadata tags that are strongly related to streaming. If you want to understand the role of all metadata, please refer to the metadata sop. Some metadata tags can refer to [alps/system/media/camera/docs/docs.html](https://alps/system/media/camera/docs/docs.html)

### 7.8.2.1 MTK\_SCALER\_AVAILABLE\_STREAM\_CONFIGURATIONS

- Description**  
The available stream configurations that this camera device supports (i.e., format, width, height, output/input stream).
- how to modify**  
For the mode you want to add, such as blob, increase the size combination you want so that you can select it in the apk.

```
CONFIG_METADATA_BEGIN(MTK_SCALER_AVAILABLE_STREAM_CONFIGURATIONS) //new
```

hidden

```
CONFIG_ENTRY_VALUE(HAL_PIXEL_FORMAT_BLOB, MINT32) //for capture
CONFIG_ENTRY_VALUE(2560, MINT32) //width
CONFIG_ENTRY_VALUE(1920, MINT32)
```



```

CONFIG_ENTRY_VALUE(MTK_SCALER_AVAILABLE_STREAM_CONFIGURATIONS_OUTPUT, MINT32)
CONFIG_ENTRY_VALUE(HAL_PIXEL_FORMAT_BLOB, MINT32) //for capture
CONFIG_ENTRY_VALUE(1920, MINT32)
CONFIG_ENTRY_VALUE(1080, MINT32)
CONFIG_ENTRY_VALUE(MTK_SCALER_AVAILABLE_STREAM_CONFIGURATIONS_OUTPUT, MINT32)
...
CONFIG_METADATA_END()

```

- ③ **Note**  
Max size combination should be larger than sensor output size combination.
- ④ **Check or warning**  
If you forget Failure to modify this may result in encountering black edges during previewing, photo capture, video recording, etc. Additionally, certain desired size combinations may not be selectable in the apk.

### 7.8.2.2 MTK\_SENSOR\_INFO\_OUTPUT\_REGION\_ON\_ACTIVE\_ARRAY

- ① **Description**  
Sensor output size info
- ② **how to modify**  
Complete the field with the maximum size combination of sensor outputs.
- ③ **note**  
This size will let the ISP know what buffer size to prepare for one frame.
- ④ **check or warning**  
If you do not fill it in correctly, streaming cannot be passed

### 7.8.2.3 MTK\_SENSOR\_INFO\_REAL\_OUTPUT\_SIZE

- ① **Description**  
sensor output size info, you can
- ② **how to modify**  
Fill in the blank with the sensor output max size combination
- ③ **note**  
The buffer size needed for one frame can be determined by this size, informing ISP about the required buffer size.
- ④ **check or warning**  
If it is not filled correctly, streaming cannot be parsed.

### 7.8.2.4 MTK\_SENSOR\_INFO\_ORIENTATION

- ① **Description**  
Clockwise angle through which the output image needs to be rotated to be upright on the device screen in its native orientation.
  - ② **how to modify**  
Degrees of clockwise rotation; always a multiple of 90
- | Orientation       | Facing Back | Facing Front |
|-------------------|-------------|--------------|
| Vertical screen   | 90          | 270          |
| Horizontal screen | 0           | 180          |
- ③ **note**  
some ICs need to modify `cfg_setting_imgsensor.cpp` orientation.  
To control the orientation for camera API2.
  - ④ **check or warning**  
If you do not fill it correctly, the image of the sensor output will have an angel issue.

### 7.8.2.5 MTK\_SENSOR\_INFO\_WANTED\_ORIENTATION

- ① **Description**  
Refer to [MTK\\_SENSOR\\_INFO\\_ORIENTATION](#)
- ② **How to modify**  
Refer to [MTK\\_SENSOR\\_INFO\\_ORIENTATION](#)
- ③ **Note**  
some ICs need to modify `cfg_setting_imgsensor.cpp` orientation. for camera api1 orientation control both MTK\_SENSOR\_INFO\_ORIENTATION or MTK\_SENSOR\_INFO\_WANTED\_ORIENTATION should be filled correctly.
- ④ **Check or warning**  
Refer to [MTK\\_SENSOR\\_INFO\\_ORIENTATION](#)

### 7.8.2.6 MTK\_SENSOR\_INFO\_FACING

- ① **Description**  
Direction The camera faces relative to the device screen.
- ② **How to modify**  
MTK\_LENS\_FACING\_BACK  
MTK\_LENS\_FACING\_FRONT  
MTK\_LENS\_FACING\_EXTERNAL  
The device should be referred to for filling.
- ③ **Note**  
Record camera hardware direction Front facing will have mirror behavior.
- ④ **Check or warning**  
Please fill it in based on your device hardware.

### 7.8.2.7 MTK\_SENSOR\_INFO\_PIXEL\_ARRAY\_SIZE

- ① **Description**

Dimensions of the full pixel array, possibly including black calibration pixels

②

**How to modify**

Fill in with sensor full size. Refer to `imgsensor_winsize_info`

③

**Note**

The pixel count of the full pixel array of the image sensor, which covers `android.sensor.info.physicalSize` area. This represents the full pixel dimensions of the raw buffers produced by this sensor. If a camera device supports raw sensor formats, either this or `android.sensor.info.preCorrectionActiveArraySize` is the maximum dimensions for the raw output formats listed in `StreamConfigurationMap` (this depends on whether or not the image sensor returns buffers containing pixels that are not part of the active array region for blacklevel calibration or other purposes). Some parts of the full pixel array may not receive light from the scene or be otherwise inactive. The `android.sensor.info.preCorrectionActiveArraySize` key defines the rectangle of active pixels that will be included in processed image formats. The pixel count of the full pixel array of the image sensor, which covers `android.sensor.info.physicalSize` area. This represents the full pixel dimensions of the raw buffers produced by this sensor.

④

**Check or warning**

Please fill it in with the size based on your register setting.

## 8 METADATA Field Associated with a Sensor

### 8.1 Exposure Related

Sensor Porting Apart from generating metadata files, it is best to verify that the following fields are properly filled in to prevent future problems with CTS porting

#### 1) File name

```
config_static_metadata.sensor.<sensorname>.h
config_static_metadata.sensor.<sensorname>.h
config_static_metadata.sensor.<sensorname>.h
config_static_metadata.sensor.<sensorname>.h
config_static_metadata.project.camera.<sensorname>.h
```

#### 2) METADATA TAG

```
MTK_SENSOR_INFO_EXPOSURE_TIME_RANGE
MTK_SENSOR_INFO_SENSITIVITY_RANGE
MTK_SENSOR_MAX_ANALOG_SENSITIVITY
MTK_SENSOR_INFO_MAX_FRAME_DURATION
MTK_SYNC_MAX_LATENCY
```

#### 3) Description

The minimum exposure time must be selected from the product of the line time of each sensor scenario and the minimum number of exposures ( $\geq$ ) whichever is the largest. Fill in according to the minimum and maximum magnifications of analog gain supported by the sensor. analog gain =  $\frac{MTK\_SENSOR\_MAX\_ANALOG\_SENSITIVITY}{1024}$  max SENSITIVITY range =  $100 * (\text{analog gain} * 1024 * 8 >> 10) / 1024 = 100 * \text{analog gain} * 8$ , This 8 times is the upper limit of ISP that AE estimates will need. Most platforms have an ISP GAIN limit of 15X, but the Noise is too high, so use 8x first. Maximum sensor analog gain support, 16x = 1600. Fill in the maximum supported exposure time, usually 400ms.

MTK\_SYNC\_MAX\_LATENCY\_UNKNOWN does not support per-frame AE control,  
MTK\_SYNC\_MAX\_LATENCY\_PER\_FRAME\_CONTROL support per-frame AE control.

#### 4) Example

```
CONFIG_METADATA_BEGIN(MTK_SENSOR_INFO_EXPOSURE_TIME_RANGE)
CONFIG_ENTRY_VALUE(100000L, MINT64)
CONFIG_ENTRY_VALUE(400000000L, MINT64)
CONFIG_METADATA_END()
if MTK_SENSOR_MAX_ANALOG_SENSITIVITY = 1600 =>
CONFIG_METADATA_BEGIN(MTK_SENSOR_INFO_SENSITIVITY_RANGE)
CONFIG_ENTRY_VALUE(100, MINT32)
CONFIG_ENTRY_VALUE(12800, MINT32)
CONFIG_METADATA_END()
CONFIG_METADATA_BEGIN(MTK_SENSOR_MAX_ANALOG_SENSITIVITY)
CONFIG_ENTRY_VALUE(1600, MINT32)
CONFIG_METADATA_END()
CONFIG_METADATA_BEGIN(MTK_SENSOR_INFO_MAX_FRAME_DURATION)
CONFIG_ENTRY_VALUE(400000000L, MINT64)
CONFIG_METADATA_END()
```

```
CONFIG_METADATA_BEGIN(MTK_SYNC_MAX_LATENCY)
CONFIG_ENTRY_VALUE(MTK_SYNC_MAX_LATENCY_UNKNOWN, MINT32)
CONFIG_METADATA_END()
```

## 8.2 FOV Related

### 1) File Name

```
config_static_metadata.sensor.<sensorname>.h
config_static_metadata.lens.<sensorname>.h
config_request_metadata_.h
```

### 2) METADATA TAG

```
MTK_SENSOR_INFO_PHYSICAL_SIZE
MTK_LENS_INFO_AVAILABLE_FOCAL_LENGTHS
MTK_LENS_FOCAL_LENGTH
```

### 3) Description

Calculate and fill in the blanks according to the Module specification.

Focal length: fill in according to Module specification.

Focal length, the same as above (VTS test).

### 4) Example

```
CONFIG_METADATA_BEGIN(MTK_SENSOR_INFO_PHYSICAL_SIZE)
    CONFIG_ENTRY_VALUE(5.28f, MFLOAT)
    CONFIG_ENTRY_VALUE(3.92f, MFLOAT)
CONFIG_METADATA_END()
CONFIG_METADATA_BEGIN(MTK_LENS_INFO_AVAILABLE_FOCAL_LENGTHS)
    CONFIG_ENTRY_VALUE(4.04f, MFLOAT)
CONFIG_METADATA_END()
CONFIG_METADATA_BEGIN(MTK_LENS_FOCAL_LENGTH)
    CONFIG_ENTRY_VALUE(5.0f, MFLOAT)
CONFIG_METADATA_END()
```

## 8.3 Sensor Size Related

The metadata folder of each platform may not be filled in, but the HalSensorList.buildStaticInfo.cpp code will fill in these fields according to the settings of the kernel driver.

### 1) File name

```
config_static_metadata.sensor.<sensorname>.h
config_static_metadata.sensor.<sensorname>.h
config_static_metadata.sensor.<sensorname>.h
```

### 2) METADATA TAG

```
MTK_SENSOR_INFO_ACTIVE_ARRAY_REGION
MTK_SENSOR_INFO_PRE_CORRECTION_ACTIVE_ARRAY_SIZE
MTK_SENSOR_INFO_PIXEL_ARRAY_SIZE
```

### 3) Description

Specify the range for the TG crop capture mode.

Fill in the sensor active pixel size.

Fill in the sensor full size.

#### 4) Example

```
CONFIG_METADATA_BEGIN(MTK_SENSOR_INFO_ACTIVE_ARRAY_REGION)
    CONFIG_ENTRY_VALUE(MRect(MPoint(0, 0), MSize(4032, 3016)), MRect)
CONFIG_METADATA_END()
```

The run time set above will be replaced by the grab size of the kernel driver capture mode in HalSensorList.buildStaticInfo.cpp.

```
.pre = {
    .pclk = 320000000, //record different mode's pclk
    .linelength = 5008, //record different mode's linelength
    .framelength = 2110, //record different mode's framelength
    .startx = 0, //record different mode's startx of grabwindow
    .starty = 0, //record different mode's starty of grabwindow
    //record different mode's width of grabwindow
    .grabwindow_width = 2104,
    //record different mode's height of grabwindow
    .grabwindow_height = 1560,
    /* by different scenario */
    .mipi_data_lp2hs_settle_dc = 85, /* unit , ns */
    /* by different scenario */
    .max_framerate = 300,
},
```

```
CONFIG_METADATA_BEGIN(MTK_SENSOR_INFO_PRE_CORRECTION_ACTIVE_ARRAY_SIZE)
    CONFIG_ENTRY_VALUE(0, MINT32)
    CONFIG_ENTRY_VALUE(0, MINT32)
    CONFIG_ENTRY_VALUE(4208, MINT32)
    CONFIG_ENTRY_VALUE(3120, MINT32)
```

```
CONFIG_METADATA_END()
```

The runtime value set above will be substituted by crop information of the kernel driver capture mode in HalSensorList.buildStaticInfo.cpp.

```
.pre = {
    .pclk = 320000000, //record different mode's pclk
    .linelength = 5008, //record different mode's linelength
    .framelength = 2110, //record different mode's framelength
    .startx = 0, //record different mode's startx of grabwindow
    .starty = 0, //record different mode's starty of grabwindow
    //record different mode's width of grabwindow
    .grabwindow_width = 2104,
    //record different mode's height of grabwindow
    .grabwindow_height = 1560,
    /* by different scenario */
    .mipi_data_lp2hs_settle_dc = 85, /* unit , ns */
    /* by different scenario */
    .max_framerate = 300,
},
```

```
CONFIG_METADATA_BEGIN(MTK_SENSOR_INFO_PIXEL_ARRAY_SIZE)
    CONFIG_ENTRY_VALUE(MSize(4208, 3120), MSize)
CONFIG_METADATA_END()
```

The above set value run time will be replaced by crop info of kernel driver capture mode in HalSensorList.buildStaticInfo.cpp.



```

/* Sensor output window information */
static struct SENSOR_WINSIZE_INFO_STRUCT imgsensor_winsize_info[5] = {
    { 4208, 3120, 0, 0, 4208, 3120, 2104, 1560,
      0000, 0000, 2104, 1560, 0, 0, 2104, 1560}, // Preview
    { 4208, 3120, 0, 0, 4208, 3120, 4208, 3120,
      0000, 0000, 4208, 3120, 0, 0, 4208, 3120}, // capture
    { 4208, 3120, 0, 0, 4208, 3120, 4208, 3120,

```

Note: HAL3 CTS has a measurement item: The active array size aspect ratio must be close to 4:3 or 16:9, which means grabwindow\_width and grabwindow\_height must be close to 4:3 or 16:9

Active array width ÷ Active array height = 1.3333 ± 0.01 for 4:3

## 8.4 Test Pattern Mode

Android Pie CTS adds the default value of test MTK\_SENSOR\_TEST\_PATTERN\_MODE, which must be set to MTK\_SENSOR\_TEST\_PATTERN\_MODE\_OFF.

### 1) File name

config\_request\_metadata\_\_.h

### 2) METADATA TAG

MTK\_SENSOR\_TEST\_PATTERN\_MODE

### 3) Description

Must be set to MTK\_SENSOR\_TEST\_PATTERN\_MODE\_OFF

### 4) Example

```

CONFIG_METADATA_BEGIN(MTK_SENSOR_TEST_PATTERN_MODE)
    CONFIG_ENTRY_VALUE(MTK_SENSOR_TEST_PATTERN_MODE_OFF, MINT32)
CONFIG_METADATA_END()

```

## 9 Unit Test

UT tests the results of the search sensor and the data of the sensor by executing the following command:

```
C:\Users> adb shell sentest

[show_Sensors]sensorNum 2
[show_Sensors]name:SENSOR_DRVNAME_OV05A20_MIPI_RAW type:0
[show_Sensors]index:0, SensorDevIdx:1
[show_Sensors]name:SENSOR_DRVNAME_IMX576_MIPI_RAW type:0
[show_Sensors]index:1, SensorDevIdx:2
[main]Param: 1 <sensorDev> <scenario> <fps>
[main]<sensorDev> : main(1), Sub(2), Main2(4), sub2(8), Main3(16)
[main]<scenario> : Pre(0), Cap(1), VD(2), slim1(3), slim2(4)
```

show that sensorNum is not 0 and display the specific name of the sensor, that is, the search for the sensor is successful, then test the output of the sensor.

```
C:\Users> adb shell sentest 1 0

[test_SensorInterface]mipi_pixel_rate = 602.807547Mpps
[test_SensorInterface]vertical_blanking = 20.508279ms
[test_SensorInterface]horizontal_blanking = 0.002407ms
[test_SensorInterface]line time = 0.006654ms
[test_SensorInterface]frame time = 33.333327ms
[test_SensorInterface]frame length = 5009.632948
[test_SensorInterface]-----
[test_SensorInterface]seninf_mux1 horizontal valid count 0x52d, horizontal blanking count
0x2f4
[test_SensorInterface]vertical valid count 0x3d0e8b, vertical blanking count 0x61a277, fps
=30
```

The vertical valid count and vertical blanking count both have data, which means that the sensor has received data.

## 10 Integration Test

Open the camera apk and test whether the screen is outputting normally, then the linearity of shutter and gain.

## Exhibit 1 Terms and Conditions

Your access to and use of this document and the information contained herein (collectively this “Document”) is subject to your (including the corporation or other legal entity you represent, collectively “You”) acceptance of the terms and conditions set forth below (“T&C”). By using, accessing or downloading this Document, You are accepting the T&C and agree to be bound by the T&C. If You don’t agree to the T&C, You may not use this Document and shall immediately destroy any copy thereof.

This Document contains information that is confidential and proprietary to MediaTek Inc. and/or its affiliates (collectively “MediaTek”) or its licensors and is provided solely for Your internal use with MediaTek’s chipset(s) described in this Document and shall not be used for any other purposes (including but not limited to identifying or providing evidence to support any potential patent infringement claim against MediaTek or any of MediaTek’s suppliers and/or direct or indirect customers). Unauthorized use or disclosure of the information contained herein is prohibited. You agree to indemnify MediaTek for any loss or damages suffered by MediaTek for Your unauthorized use or disclosure of this Document, in whole or in part.

MediaTek and its licensors retain titles and all ownership rights in and to this Document and no license (express or implied, by estoppels or otherwise) to any intellectual propriety rights is granted hereunder. This Document is subject to change without further notification. MediaTek does not assume any responsibility arising out of or in connection with any use of, or reliance on, this Document, and specifically disclaims any and all liability, including, without limitation, consequential or incidental damages.

THIS DOCUMENT AND ANY OTHER MATERIALS OR TECHNICAL SUPPORT PROVIDED BY MEDIATEK IN CONNECTION WITH THIS DOCUMENT, IF ANY, ARE PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE. MEDIATEK SPECIFICALLY DISCLAIMS ALL WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, COMPLETENESS OR ACCURACY AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MEDIATEK SHALL NOT BE RESPONSIBLE FOR ANY MEDIATEK DELIVERABLES MADE TO MEET YOUR SPECIFICATIONS OR TO CONFORM TO A PARTICULAR STANDARD OR OPEN FORUM.

Without limiting the generality of the foregoing, MediaTek makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does MediaTek assume any liability arising out of the application or use of any product, circuit or software. You agree that You are solely responsible for the designing, validating and testing Your product incorporating MediaTek’s product and ensure such product meets applicable standards and any safety, security or other requirements.

The above T&C and all acts in connection with the T&C or this Document shall be governed, construed and interpreted in accordance with the laws of Taiwan, without giving effect to the principles of conflicts of law.